

```

F01:0001 """EgoSmplx: automatic fisheye body fitting and fixed-rule MANO fusion."""
F01:0002 __version__ = "1.0.0"
F02:0001 """Command-line interface: check, run, sample."""
F02:0002 import argparse
F02:0003 from pathlib import Path
F02:0004 from egosmplx_pipeline.configuration import load_config, preflight
F02:0005 from egosmplx_pipeline.io import write
F02:0008 def main():
F02:0009     parser = argparse.ArgumentParser(description='EgoSmplx V1.0 automatic fisheye pose pipeline')
F02:0010     sub = parser.add_subparsers(dest='command', required=True)
F02:0011     for name in ['check', 'run']:
F02:0012         cmd = sub.add_parser(name)
F02:0013         cmd.add_argument('--config', type=Path, required=True)
F02:0014         cmd.add_argument('--cached-predictions', action='store_true')
F02:0015         if name == 'run':
F02:0016             cmd.add_argument('--resume', action='store_true')
F02:0017     sample = sub.add_parser('sample')
F02:0018     sample.add_argument('--video', type=Path, required=True)
F02:0019     sample.add_argument('--calibration', type=Path, required=True)
F02:0020     sample.add_argument('--camera', required=True)
F02:0021     sample.add_argument('--count', type=int, default=4)
F02:0022     sample.add_argument('--output', type=Path, required=True)
F02:0023     args = parser.parse_args()
F02:0024     if args.command == 'check':
F02:0025         config, rows = load_config(args.config)
F02:0026         print(preflight(config, rows, args.cached_predictions))
F02:0027     elif args.command == 'run':
F02:0028         from egosmplx_pipeline.pipeline import run
F02:0029         print(run(args.config, args.cached_predictions, args.resume))
F02:0030     else:
F02:0031         from egosmplx_pipeline.sampling import sample_video
F02:0032         print(sample_video(args.video, args.calibration, args.camera, args.count, args.output))
F02:0035 if __name__ == '__main__':
F02:0036     main()
F03:0001 """Portable JSON configuration and fail-fast input checks."""
F03:0002 import os
F03:0003 import re
F03:0004 from pathlib import Path
F03:0005 from egosmplx_pipeline.io import read, sha
F03:0007 PACKAGE_ROOT = Path(__file__).resolve().parents[1]
F03:0010 def expand(value, base):
F03:0011     if isinstance(value, dict):
F03:0012         return {k: expand(v, base) for k, v in value.items()}
F03:0013     if isinstance(value, list):
F03:0014         return [expand(v, base) for v in value]
F03:0015     if isinstance(value, str):
F03:0016         value = value.replace('${PACKAGE_ROOT}', str(PACKAGE_ROOT))
F03:0017         value = os.path.expandvars(value)
F03:0018         if re.search(r'\${[^\}]+\}', value):
F03:0019             raise ValueError('Unresolved environment variable: ' + value)

```

```

F03:0020     return value
F03:0023 def absolute(path, base):
F03:0024     p = Path(path).expanduser()
F03:0025     return str((base / p).resolve() if not p.is_absolute() else p.resolve())
F03:0028 def load_config(path):
F03:0029     path = Path(path).resolve()
F03:0030     config = expand(read(path), path.parent)
F03:0031     for key, value in config['paths'].items():
F03:0032         if value:
F03:0033             config['paths'][key] = absolute(value, path.parent)
F03:0034     config['manifest'] = absolute(config['manifest'], path.parent)
F03:0035     config['output'] = absolute(config['output'], path.parent)
F03:0036     fit = config.setdefault('fitting', {})
F03:0037     for key, default in [('rigid_steps', 1800), ('pose_steps', 1600), ('silhouette_weight', .08)]:
F03:0038         fit.setdefault(key, default)
F03:0039     if min(fit['rigid_steps'], fit['pose_steps']) < 1 or fit['silhouette_weight'] < 0:
F03:0040         raise ValueError('Invalid fitting settings')
F03:0041     manifest = read(config['manifest'])
F03:0042     frames = expand(manifest['frames'], Path(config['manifest']).parent)
F03:0043     ids = set()
F03:0044     for row in frames:
F03:0045         identifier = row['id']
F03:0046         if not re.fullmatch(r'[A-Za-z0-9_-]+', identifier) or identifier in ids:
F03:0047             raise ValueError('Frame IDs must be unique safe filenames: ' + identifier)
F03:0048         ids.add(identifier)
F03:0049         for key in ['image', 'calibration']:
F03:0050             row[key] = absolute(row[key], Path(config['manifest']).parent)
F03:0051         for key, value in row.get('predictions', {}).items():
F03:0052             row['predictions'][key] = absolute(value, Path(config['manifest']).parent)
F03:0053     if not frames:
F03:0054         raise ValueError('Empty input manifest')
F03:0055     return config, frames
F03:0058 def preflight(config, frames, cached):
F03:0059     """Check required files and image/observation provenance before any output."""
F03:0060     import cv2
F03:0061     required = ['body_python', 'egosmplx_repo', 'egosmplx_config']
F03:0062     if not cached:
F03:0063         required += ['pose_python', 'wilor_python', 'sapiens_repo', 'wilor_repo',
F03:0064                     'egosmplx_checkpoint', 'pose_checkpoint', 'seg_checkpoint',
F03:0065                     'wilor_checkpoint', 'wilor_config', 'wilor_detector']
F03:0066     for key in required:
F03:0067         if key not in config['paths'] or not Path(config['paths'][key]).exists():
F03:0068             raise FileNotFoundError('Required configured path: ' + key)
F03:0069     for row in frames:
F03:0070         image = cv2.imread(row['image'])
F03:0071         if image is None:
F03:0072             raise ValueError('Unreadable image: ' + row['image'])
F03:0073         calibration = read(row['calibration'])
F03:0074         if calibration['size'] != list(image.shape[1::-1]):
F03:0075             raise ValueError('Image and calibration sizes differ: ' + row['id'])

```

```

F03:0076         if cached:
F03:0077             for key in ['egosmplx', 'sapiens', 'segmentation', 'wilor']:
F03:0078                 if not Path(row.get('predictions', {})).get(key, '').exists() or key not in row.get(
F03:0078+ 'predictions', {}):
F03:0079                     raise FileNotFoundError(row['id'] + ': missing cached prediction ' + key)
F03:0080                 points = read(row['predictions']['sapiens'])
F03:0081                 if points['image_sha256'] != sha(row['image']):
F03:0082                     raise ValueError('Stale Sapiens2 observation: ' + row['id'])
F03:0083                 for name in ['labels.npy', 'probabilities.npz']:
F03:0084                     if not (Path(row['predictions']['segmentation']) / name).is_file():
F03:0085                         raise FileNotFoundError('Incomplete segmentation: ' + row['id'])
F03:0086         if not cached:
F03:0087             for name in ['pose', 'seg']:
F03:0088                 expected = config.get('checkpoint_sha256', {}).get(name)
F03:0089                 if expected is None or not re.fullmatch('[a-f0-9]{64}', expected):
F03:0090                     raise ValueError('Configure checkpoint_sha256.' + name)
F03:0091         return dict(passed=True, frame_count=len(frames), cached_predictions=cached,
F03:0092                     scope='paths, input sizes, cache image hashes; models are loaded strictly by workers
F03:0092+ ')
F04:0001 """Sequential multi-environment controller with input hashing and verified resume."""
F04:0002 import copy
F04:0003 import datetime
F04:0004 import fcntl
F04:0005 import hashlib
F04:0006 import os
F04:0007 from pathlib import Path
F04:0008 import shutil
F04:0009 import subprocess
F04:0010 import sys
F04:0011 from egosmplx_pipeline.configuration import PACKAGE_ROOT, load_config, preflight
F04:0012 from egosmplx_pipeline.io import read, write, sha
F04:0015 def command(args, log, cwd, environment):
F04:0016     print('RUN', log.name, flush=True)
F04:0017     with log.open('w') as stream:
F04:0018         process = subprocess.run(list(map(str, args)), cwd=str(cwd), env=environment,
F04:0019                                 stdout=stream, stderr=subprocess.STDOUT)
F04:0020     if process.returncode:
F04:0021         raise RuntimeError('Worker failed; inspect ' + str(log))
F04:0024 def environment(config):
F04:0025     result = os.environ.copy()
F04:0026     result.update(OMP_NUM_THREADS='2', MKL_NUM_THREADS='2', PYTHONDONTWRITEBYTECODE='1',
F04:0027                  CUDA_VISIBLE_DEVICES=str(config.get('gpu', '0')),
F04:0028                  EGOSMPLX_CONFIG=config['paths']['egosmplx_config'],
F04:0029                  EGOSMPLX_REPO_ROOT=config['paths']['egosmplx_repo'])
F04:0030     result['PYTHONPATH'] = os.pathsep.join([str(PACKAGE_ROOT), config['paths'].get('wilor_repo', '')
F04:0030+ ])
F04:0031     return result
F04:0034 def observe(config, rows, output, env):
F04:0035     """Invoke independent external networks. No manual annotation input."""
F04:0036     p = config['paths']

```

```

F04:0037     observations = output / 'observations'
F04:0038     observations.mkdir(exist_ok=False)
F04:0039     staged = observations / 'images'
F04:0040     staged.mkdir()
F04:0041     observed_rows = copy.deepcopy(rows)
F04:0042     for row in observed_rows:
F04:0043         image = Path(row['image'])
F04:0044         target = staged / (row['id'] + image.suffix.lower())
F04:0045         shutil.copy2(image, target)
F04:0046         row['image'] = str(target)
F04:0047     manifest = observations / 'manifest.json'
F04:0048     write(manifest, dict(frames=observed_rows))
F04:0049     for row in observed_rows:
F04:0050         directory = observations / row['id']
F04:0051         directory.mkdir()
F04:0052         command([p['body_python'], '-m', 'egosmplx_pipeline.egosmplx_predict', '--image-dir', staged
F04:0052+ , '--image-glob', Path(row['image']).name,
F04:0053                     '--fisheye-calibration', row['calibration'], '--config-path', p['egosmplx_config'],
F04:0054                     '--checkpoint-path', p['egosmplx_checkpoint'], '--output-dir', directory / 'network
F04:0054+ ',
F04:0055                     '--smplx-input-mode', 'det_undistort_contiguous', '--contiguous-roii-mode', 'aspect_
F04:0055+ pad'],
F04:0056                     directory / 'network.log', p['egosmplx_repo'], env)
F04:0057         candidates = sorted((directory / 'network').rglob(row['id'] + '*_smplx.npz'))
F04:0058         # Some backend versions return the timestamp directory beside the request.
F04:0059         if not candidates:
F04:0060             candidates = sorted(directory.glob('network*/smplx/' + row['id'] + '*_smplx.npz'))
F04:0061         if len(candidates) != 1:
F04:0062             raise RuntimeError('Expected one wearer prediction, found ' + str(len(candidates)))
F04:0063         args = [p['pose_python'], '-m', 'egosmplx_pipeline.sapiens_pose', '--repo-root', p['sapiens_
F04:0063+ repo'],
F04:0064                     '--image', row['image'], '--output-dir', directory / 'sapiens',
F04:0065                     '--checkpoint', p['pose_checkpoint'], '--checkpoint-sha256', config['checkpoint_sha2
F04:0065+ 56']['pose']]
F04:0066         if p.get('sapiens_dependencies'):
F04:0067             args += ['--dependencies', p['sapiens_dependencies']]
F04:0068         command(args, directory / 'sapiens.log', PACKAGE_ROOT, env)
F04:0069         row['predictions'] = dict(egosmplx=str(candidates[0]), sapiens=str(directory / 'sapiens/keyp
F04:0069+ oints.json'))
F04:0070         command([p['wilor_python'], '-m', 'egosmplx_pipeline.wilor_predict', '--manifest', manifest,
F04:0071                     '--output-dir', observations / 'wilor', '--checkpoint', p['wilor_checkpoint'],
F04:0072                     '--config', p['wilor_config'], '--detector', p['wilor_detector']],
F04:0073                     observations / 'wilor.log', p['wilor_repo'], env)
F04:0074         args = [p['pose_python'], '-m', 'egosmplx_pipeline.sapiens_seg', '--repo-root', p['sapiens_repo'
F04:0074+ ],
F04:0075                     '--manifest', manifest, '--output-dir', observations / 'segmentation',
F04:0076                     '--checkpoint', p['seg_checkpoint'], '--checkpoint-sha256', config['checkpoint_sha256'] [
F04:0076+ 'seg']]
F04:0077         if p.get('sapiens_dependencies'):
F04:0078             args += ['--dependencies', p['sapiens_dependencies']]

```

```

F04:0079     command(args, observations / 'segmentation.log', PACKAGE_ROOT, env)
F04:0080     for row in observed_rows:
F04:0081         row['predictions'].update(wilor=str(observations / 'wilor/npz' / (row['id'] + '_wilor.npz'))
F04:0081+ ,
F04:0082                                     segmentation=str(observations / 'segmentation/frames' / row['id'])
F04:0082+ )
F04:0083     write(output / 'automatic_manifest.json', dict(frames=observed_rows))
F04:0084     return observed_rows
F04:0087 def fingerprint(config, rows):
F04:0088     import json
F04:0089     hashes = {}
F04:0090     for row in rows:
F04:0091         for key in ['image', 'calibration']:
F04:0092             hashes[row['id'] + ':' + key] = sha(row[key])
F04:0093         for key, value in row.get('predictions', {}).items():
F04:0094             if key == 'segmentation':
F04:0095                 for name in ['labels.npy', 'probabilities.npz']:
F04:0096                     hashes[row['id'] + ':' + name] = sha(Path(value) / name)
F04:0097             else:
F04:0098                 hashes[row['id'] + ':' + key] = sha(value)
F04:0099     code = {str(p.relative_to(PACKAGE_ROOT)): sha(p) for p in (PACKAGE_ROOT / 'egosmplx_pipeline').g
F04:0099+ lob('*.py')}
F04:0100     return hashlib.sha256(json.dumps(dict(config=config, inputs=hashes, code=code), sort_keys=True).
F04:0100+ encode()).hexdigest()
F04:0103 def run(config_path, cached=False, resume=False):
F04:0104     config, rows = load_config(config_path)
F04:0105     check = preflight(config, rows, cached)
F04:0106     output = Path(config['output'])
F04:0107     if output.exists() and not resume:
F04:0108         raise FileExistsError('Output exists: choose a new directory or --resume')
F04:0109     output.mkdir(parents=True, exist_ok=True)
F04:0110     with (output / '.lock').open('w') as lock:
F04:0111         fcntl.flock(lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
F04:0112         env = environment(config)
F04:0113         write(output / 'preflight.json', check)
F04:0114         signature = fingerprint(config, rows)
F04:0115         state_path = output / 'run_state.json'
F04:0116         if state_path.exists() and read(state_path)['fingerprint'] != signature:
F04:0117             raise ValueError('Inputs, code, or configuration changed: use a new output directory')
F04:0118         write(state_path, dict(fingerprint=signature, state='running', cached_predictions=cached))
F04:0119         if not cached:
F04:0120             ready = output / 'automatic_manifest.json'
F04:0121             if resume and ready.exists():
F04:0122                 rows = read(ready)['frames']
F04:0123             else:
F04:0124                 if (output / 'observations').exists():
F04:0125                     stamp = datetime.datetime.now().strftime('%Y%m%dT%H%M%S%f')
F04:0126                     (output / 'observations').rename(output / ('incomplete_observations_' + stamp))
F04:0127                 rows = observe(config, rows, output, env)
F04:0128     records = []

```

```

F04:0129         for index, row in enumerate(rows):
F04:0130             destination = output / 'frames' / row['id']
F04:0131             validation = destination / 'validation.json'
F04:0132             if resume and validation.exists():
F04:0133                 report = read(validation)
F04:0134                 valid = report.get('passed') and all((destination / f).is_file() and sha(destination
F04:0134+ / f) == h
F04:0135                                     for f, h in report['artifact_sha256'].items())
F04:0136                 if valid:
F04:0137                     records.append(report)
F04:0138                     print('VERIFIED_RESUME', row['id'], flush=True)
F04:0139                     continue
F04:0140             if destination.exists():
F04:0141                 stamp = datetime.datetime.now().strftime('%Y%m%dT%H%M%S%f')
F04:0142                 destination.rename(destination.with_name(destination.name + '_incomplete_' + stamp))
F04:0143             job_path = output / 'jobs' / (row['id'] + '.json')
F04:0144             write(job_path, dict(config=config, row=row, output=str(destination)))
F04:0145             command([config['paths'] ['body_python'], '-m', 'egosmplx_pipeline.frame', '--job', job_p
F04:0145+ ath],
F04:0146                     output / (row['id'] + '.log'), PACKAGE_ROOT, env)
F04:0147             records.append(read(validation))
F04:0148             write(output / 'progress.json', dict(completed=index+1, total=len(rows), last_frame=row[
F04:0148+ 'id']))
F04:0149             from egosmplx_pipeline.report import build
F04:0150             build(output, records)
F04:0151             write(state_path, dict(fingerprint=signature, state='complete', frame_count=len(records), ca
F04:0151+ ched_predictions=cached))
F04:0152         return output
F05:0001 """Execute fitting and fixed-rule fusion for one automatically observed frame."""
F05:0002 import argparse
F05:0003 import json
F05:0004 from pathlib import Path
F05:0005 import sys
F05:0006 import cv2
F05:0007 import numpy as np
F05:0008 from egosmplx_pipeline.io import read, write, load, sha, rmse
F05:0011 def canonicalize(parameters, model, camera):
F05:0012     from egosmplx_pipeline.fusion import geometry
F05:0013     vertices, joints = geometry(parameters)
F05:0014     pixels = camera.world2camera(joints.astype(np.float64)).astype(np.float32)
F05:0015     parameters.update(smplx_mesh_cam=vertices, smplx_joint_cam=joints, smplx_joint_img=pixels,
F05:0016                     smplx_joint_cam_root_relative=joints-joints[:1],
F05:0017                     smplx_body_joints_3d_local=(joints-parameters['transl']).reshape(3))[:25],
F05:0018                     smplx_body_joints_3d_render_cam=joints[:25],
F05:0019                     smplx_body_joints_2d_render_cam=pixels[:25])
F05:0020     return parameters
F05:0023 def invoke(function, arguments):
F05:0024     previous = sys.argv
F05:0025     try:
F05:0026         sys.argv = [function.__module__] + list(map(str, arguments))

```

```

F05:0027         function()
F05:0028     finally:
F05:0029         sys.argv = previous
F05:0032 def run(config, row, output):
F05:0033     from egosmplx_pipeline.bootstrap import setup
F05:0034     model, Camera = setup(config['paths']['egosmplx_repo'])
F05:0035     from egosmplx_pipeline import fusion, body_rigid, body_pose
F05:0036     from egosmplx_pipeline.matching import assign
F05:0037     from egosmplx_pipeline.render import render, write_obj, read_obj
F05:0038     fusion.initialize(model)
F05:0039     camera = Camera(row['calibration'])
F05:0040     image = cv2.imread(row['image'])
F05:0041     if image is None or tuple(camera.img_size) != tuple(image.shape[1::-1]):
F05:0042         raise ValueError('Image or calibration dimensions invalid')
F05:0043     output.mkdir(parents=True, exist_ok=False)
F05:0044     predictions = row['predictions']
F05:0045     targets = read(predictions['sapiens'])
F05:0046     image_digest = sha(row['image'])
F05:0047     if targets['image_sha256'] != image_digest:
F05:0048         raise ValueError('Automatic target image hash differs from input')
F05:0049     raw = load(predictions['egosmplx'])
F05:0050     for key in ['global_orient', 'transl']:
F05:0051         if key + '_network' not in raw:
F05:0052             raise ValueError('Raw prediction requires explicit original network field: ' + key)
F05:0053             raw[key] = raw[key + '_network'].copy()
F05:0054     # Retain reconstruction inputs and original network rigid outputs only.
F05:0055     keep = fusion.PARAMS + ['transl', 'global_orient_network', 'transl_network']
F05:0056     raw = canonicalize({k: raw[k] for k in keep}, model, camera)
F05:0057     raw_dir = output / 'raw'
F05:0058     raw_dir.mkdir()
F05:0059     np.savez_compressed(raw_dir / 'params.npz', **raw)
F05:0060     write_obj(raw_dir / 'mesh.obj', raw['smplx_mesh_cam'], model.face)
F05:0061     cv2.imwrite(str(raw_dir / 'wireframe.jpg'), render(image, raw['smplx_mesh_cam'].astype(np.float6
F05:0061+ 4), model.face, camera, .12))
F05:0062     common = ['--repo-root', config['paths']['egosmplx_repo'], '--image', row['image'], '--calibrati
F05:0062+ on', row['calibration']]
F05:0063     invoke(body_rigid.main, common + ['--source-npz', raw_dir / 'params.npz', '--keypoints', predict
F05:0063+ ions['sapiens'],
F05:0064         '--output-dir', output / 'rigid', '--steps', config['fitting']['
F05:0064+ rigid_steps'], '--device', 'cpu'])
F05:0065     invoke(body_pose.main, common + ['--source-npz', output / 'rigid/optimized_params.npz',
F05:0066         '--automatic-keypoints', predictions['sapiens'], '--segmentation'
F05:0066+ , predictions['segmentation'],
F05:0067         '--output-dir', output / 'body_fit', '--steps', config['fitting']
F05:0067+ ['pose_steps'],
F05:0068         '--silhouette-weight', config['fitting']['silhouette_weight']])
F05:0069     body = canonicalize(load(output / 'body_fit/automatic/refined_params.npz'), model, camera)
F05:0070     body_dir = output / 'body'
F05:0071     body_dir.mkdir()
F05:0072     np.savez_compressed(body_dir / 'params.npz', **body)

```

```

F05:0073     write_obj(body_dir / 'mesh.obj', body['smp1x_mesh_cam'], model.face)
F05:0074     cv2.imwrite(str(body_dir / 'wireframe.jpg'), render(image, body['smp1x_mesh_cam'].astype(np.floa
F05:0074+ t64), model.face, camera, .12))
F05:0075     hands = load(predictions['wilor'])
F05:0076     if 'image_sha256' in hands and str(hands['image_sha256']) != image_digest:
F05:0077         raise ValueError('WiLoR image hash differs from input')
F05:0078     if 'focal_length_px' not in hands:
F05:0079         if 'cached_wilor_focal_px' not in config:
F05:0080             raise ValueError('Legacy WiLoR cache requires an explicit verified focal length')
F05:0081             hands['focal_length_px'] = np.array(config['cached_wilor_focal_px'])
F05:0082             hands['image_size_wh'] = np.array(image.shape[1::-1])
F05:0083     if not np.array_equal(hands['image_size_wh'], image.shape[1::-1]):
F05:0084         raise ValueError('WiLoR image size mismatch')
F05:0085     lookup = {q['name']: q for q in targets['keypoints']}
F05:0086     matches = assign(hands, lookup, body['smp1x_joint_img'], *image.shape[1::-1])
F05:0087     mesh, hand_reports = fusion.stitch(body, hands, matches, camera)
F05:0088     fused = output / 'fused'
F05:0089     fused.mkdir()
F05:0090     np.savez_compressed(fused / 'mesh.npz', **mesh)
F05:0091     np.savez_compressed(fused / 'wilor_predictions.npz', **hands)
F05:0092     write_obj(fused / 'mesh.obj', mesh['vertices_cam'], mesh['faces'])
F05:0093     write(fused / 'recipe.json', dict(matches=matches, blend_rings=7, body_parameter_path='../body/p
F05:0093+ arams.npz',
F05:0094                                     manual_annotations_used=False, local_surface_optimizer=False,
F05:0095                                     standard_smp1x_parameter_only_reconstruction=False))
F05:0096     cv2.imwrite(str(fused / 'wireframe.jpg'), render(image, mesh['vertices_cam'].astype(np.float64),
F05:0096+ mesh['faces'], camera, .12))
F05:0097     cv2.imwrite(str(fused / 'skeleton.jpg'), fusion.draw_geometry(image, camera, mesh))
F05:0098     # Reload actual saved artifacts, including the fusion inputs and recipe.
F05:0099     reloaded_body = load(body_dir / 'params.npz')
F05:0100     v, j = fusion.geometry(reloaded_body)
F05:0101     body_error = float(np.max(np.abs(v-reloaded_body['smp1x_mesh_cam'])))
F05:0102     rebuilt, _ = fusion.stitch(reloaded_body, load(fused / 'wilor_predictions.npz'), read(fused / 'r
F05:0102+ ecipe.json')['matches'], camera)
F05:0103     stored = load(fused / 'mesh.npz')
F05:0104     fusion_error = float(np.max(np.abs(rebuilt['vertices_cam']-stored['vertices_cam'])))
F05:0105     obj_vertices, obj_faces = read_obj(fused / 'mesh.obj')
F05:0106     obj_error = float(np.max(np.abs(obj_vertices-stored['vertices_cam'])))
F05:0107     if max(body_error, fusion_error, obj_error) > 1e-5 or not np.array_equal(obj_faces, stored['face
F05:0107+ s']):
F05:0108         raise RuntimeError('Saved geometry failed reload validation')
F05:0109     indices = body['stage2_selected_joint_indices'].astype(int)
F05:0110     truth = body['stage2_target_joint_pixels']
F05:0111     body_errors = np.linalg.norm(body['smp1x_joint_img'][indices]-truth, axis=1)
F05:0112     hand_errors = []
F05:0113     for side, match in matches.items():
F05:0114         k = fusion.SIDES.index(side)
F05:0115         projected = camera.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
F05:0116         hand_errors.extend(np.linalg.norm(projected-hands['joints_2d'][match['candidate']], axis=1).
F05:0116+ tolist())

```



```

F05:0117     report = dict(id=row['id'], passed=True, matched_hands=len(matches), matches=matches,
F05:0118         body_target_count=len(indices), body_error_px=body_errors.tolist(), hand_error_px=
F05:0118+ hand_errors,
F05:0119         body_rmse_px=rmse(body['smplx_joint_img'][indices], truth),
F05:0120         hand_rmse_px=float(np.sqrt(np.mean(np.square(hand_errors)))) if hand_errors else N
F05:0120+ one,
F05:0121         body_reload_max_abs_m=body_error, fusion_reload_max_abs_m=fusion_error,
F05:0122         obj_reload_max_abs_m=obj_error, minimum_fused_vertex_z_m=float(mesh['vertices_cam'
F05:0122+ ][:,2].min()),
F05:0123         hand_reports=hand_reports, manual_annotations_used=False,
F05:0124         accuracy_reference='automatic predictions, not human ground truth',
F05:0125         input_image_sha256=image_digest, calibration_sha256=sha(row['calibration']))
F05:0126     report['artifact_sha256'] = {str(p.relative_to(output)): sha(p) for p in output.rglob('*') if p.
F05:0126+ is_file()}
F05:0127     write(output / 'validation.json', report)
F05:0128     print('FRAME_COMPLETE', row['id'], report['body_rmse_px'], report['hand_rmse_px'], flush=True)
F05:0129     return report
F05:0132 def main():
F05:0133     parser = argparse.ArgumentParser(description=__doc__)
F05:0134     parser.add_argument('--job', type=Path, required=True)
F05:0135     args = parser.parse_args()
F05:0136     job = read(args.job)
F05:0137     run(job['config'], job['row'], Path(job['output']))
F05:0140 if __name__ == '__main__':
F05:0141     main()
F06:0001 """Initialize the separately installed EgoSMPLX backend in a worker process."""
F06:0002 import os
F06:0003 import sys
F06:0004 from pathlib import Path
F06:0006 def setup(repo):
F06:0007     repo = Path(repo).resolve()
F06:0008     for sub in ['', 'main', 'common', 'data']:
F06:0009         sys.path.insert(0, str(repo / sub))
F06:0010     os.environ['EGOSMPLX_REPO_ROOT'] = str(repo)
F06:0011     from config import cfg
F06:0012     path = os.environ.get('EGOSMPLX_CONFIG', str(repo / 'main/config/config_ft_egopw_sceneego_egowho
F06:0012+ 1body_unrealego_egofishbody.py'))
F06:0013     cfg.get_config_fromfile(path)
F06:0014     from egosmplx_pipeline import camera as calibrated_camera
F06:0015     sys.modules['FishEyeCalibrated'] = calibrated_camera
F06:0016     from egosmplx_pipeline.camera import FishEyeCameraCalibrated
F06:0017     from egosmplx_pipeline.projection import stable_world2camera
F06:0018     FishEyeCameraCalibrated.world2camera_pytorch = stable_world2camera
F06:0019     from utils.human_models import smplx
F06:0020     import torch
F06:0021     torch.set_num_threads(2)
F06:0022     torch.backends.cuda.matmul.allow_tf32 = False
F06:0023     torch.backends.cudnn.allow_tf32 = False
F06:0024     return smplx, FishEyeCameraCalibrated
F07:0001 """Strict JSON/NPZ I/O and reproducibility metadata."""

```

```

F07:0002 import hashlib
F07:0003 import json
F07:0004 from pathlib import Path
F07:0005 import numpy as np
F07:0008 def read(path):
F07:0009     return json.loads(Path(path).read_text(encoding='utf-8'))
F07:0012 def write(path, value):
F07:0013     path = Path(path)
F07:0014     path.parent.mkdir(parents=True, exist_ok=True)
F07:0015     temporary = path.with_suffix(path.suffix + '.tmp')
F07:0016     temporary.write_text(json.dumps(value, indent=2, ensure_ascii=False, allow_nan=False) + '\n', en
F07:0016+ coding='utf-8')
F07:0017     temporary.replace(path)
F07:0020 def sha(path):
F07:0021     h = hashlib.sha256()
F07:0022     with Path(path).open('rb') as handle:
F07:0023         for block in iter(lambda: handle.read(8 * 1024**2), b''):
F07:0024             h.update(block)
F07:0025     return h.hexdigest()
F07:0028 def load(path):
F07:0029     with np.load(path, allow_pickle=False) as data:
F07:0030         result = {k: data[k].copy() for k in data.files}
F07:0031     for key, value in result.items():
F07:0032         if np.issubdtype(value.dtype, np.number) and not np.isfinite(value).all():
F07:0033             raise ValueError('Nonfinite NPZ field: ' + key)
F07:0034     return result
F07:0037 def arr(tensor):
F07:0038     return tensor.detach().cpu().numpy()
F07:0041 def rmse(prediction, target):
F07:0042     """Root mean squared 2D Euclidean distance, not coordinate-wise RMSE."""
F07:0043     prediction, target = np.asarray(prediction), np.asarray(target)
F07:0044     if prediction.shape != target.shape or prediction.ndim != 2 or prediction.shape[1] != 2:
F07:0045         raise ValueError('Expected matching N x 2 arrays')
F07:0046     if not len(prediction):
F07:0047         return None
F07:0048     return float(np.sqrt(np.mean(np.sum((prediction - target)**2, axis=1))))
F08:0001 """Decode evenly spaced video frames; do not claim stereo synchronization."""
F08:0002 from pathlib import Path
F08:0003 import re
F08:0004 import cv2
F08:0005 from egosmplx_pipeline.io import write, sha
F08:0008 def sample_video(video, calibration, camera, count, output):
F08:0009     if count < 1 or not re.fullmatch('[A-Za-z0-9_-]+', camera):
F08:0010         raise ValueError('Positive frame count and safe camera name required')
F08:0011     output = Path(output)
F08:0012     if output.exists():
F08:0013         raise FileExistsError(output)
F08:0014     cap = cv2.VideoCapture(str(video))
F08:0015     try:
F08:0016         n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

```

```

F08:0017         fps = cap.get(cv2.CAP_PROP_FPS)
F08:0018         if not cap.isOpened() or n < count or fps <= 0:
F08:0019             raise ValueError('Video cannot provide requested frames')
F08:0020         output.mkdir(parents=True)
F08:0021         rows = []
F08:0022         for k in range(count):
F08:0023             index = min(n-1, int((k+.5) * n/count))
F08:0024             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
F08:0025             ok, image = cap.read()
F08:0026             if not ok:
F08:0027                 raise RuntimeError('Failed decoding frame ' + str(index))
F08:0028             identifier = camera + '_f' + str(index).zfill(8)
F08:0029             target = output / (identifier + '.png')
F08:0030             if not cv2.imwrite(str(target), image):
F08:0031                 raise IOError(target)
F08:0032             rows.append(dict(id=identifier, camera=camera, image=target.name,
F08:0033                             calibration=str(Path(calibration).resolve()), image_sha256=sha(target),
F08:0034                             frame_index_zero_based=index, approximate_video_seconds=index/fps))
F08:0035         write(output / 'manifest.json', dict(frames=rows, exact_stereo_synchronization_claimed=False
F08:0035+ ))
F08:0036         return output / 'manifest.json'
F08:0037     finally:
F08:0038         cap.release()
F09:0001 #!/usr/bin/env python3
F09:0002 """Single-person Sapiens2 inference using the installed official pipeline.
F09:0004 An explicit box or the full image is used; no manual keypoint annotations are
F09:0005 read. Output keypoints are in original image pixels, with uncalibrated scores.
F09:0006 """
F09:0007 import argparse
F09:0008 import hashlib
F09:0009 import json
F09:0010 import sys
F09:0011 import time
F09:0012 from pathlib import Path
F09:0013 def main():
F09:0014     p = argparse.ArgumentParser(description=__doc__)
F09:0015     p.add_argument('--repo-root', type=Path, required=True, help='Sapiens2 checkout')
F09:0016     p.add_argument('--dependencies', type=Path)
F09:0017     p.add_argument('--checkpoint', type=Path, required=True)
F09:0018     p.add_argument('--checkpoint-sha256', required=True)
F09:0019     p.add_argument('--image', type=Path, required=True)
F09:0020     p.add_argument('--output-dir', type=Path, required=True)
F09:0021     p.add_argument('--size', choices=['1b', '5b'], default='1b')
F09:0022     p.add_argument('--bbox', nargs=4, type=float, help='Original image xxyy; default full image')
F09:0023     a = p.parse_args()
F09:0024     root = a.repo_root.resolve()
F09:0025     repo = root
F09:0026     sys.path.insert(0, str(repo))
F09:0027     if a.dependencies:
F09:0028         sys.path.insert(0, str(a.dependencies))

```

```

F09:0031     import cv2
F09:0032     import numpy as np
F09:0033     import torch
F09:0034     from safetensors.torch import load_file
F09:0035     from sapiens.engine.config import Config
F09:0036     from sapiens.engine.datasets import Compose
F09:0037     from sapiens.pose.datasets import parse_pose_metainfo
F09:0038     from sapiens.pose.datasets.codecs.udp_heatmap import UDPHeatmap
F09:0039     from sapiens.registry import MODELS
F09:0041     im = cv2.imread(str(a.image))
F09:0042     if im is None:
F09:0043         raise ValueError(f'Unreadable image: {a.image}')
F09:0044     h, w = im.shape[:2]
F09:0045     bbox = a.bbox or [0, 0, w, h]
F09:0046     if not (0 <= bbox[0] < bbox[2] <= w and 0 <= bbox[1] < bbox[3] <= h):
F09:0047         raise ValueError('bbox must lie inside the original image')
F09:0048     a.output_dir.mkdir(parents=True, exist_ok=False)
F09:0049     weight = a.checkpoint
F09:0050     digest = hashlib.sha256()
F09:0051     with weight.open('rb') as f:
F09:0052         for block in iter(lambda: f.read(8 * 1024**2), b''):
F09:0053             digest.update(block)
F09:0054     if digest.hexdigest() != a.checkpoint_sha256:
F09:0055         raise ValueError('Pose checkpoint SHA-256 mismatch')
F09:0056     config = repo / f'sapiens/pose/configs/keypoints308/shutterstock_goliath_3po/sapiens2_{a.size}_k
F09:0056+ eypoints308_shutterstock_goliath_3po-1024x768.py'
F09:0057     cfg = Config.fromfile(config)
F09:0058     cfg.model['backbone'].pop('init_cfg', None)
F09:0059     print('Building official pose model', flush=True)
F09:0060     model = MODELS.build(cfg.model)
F09:0061     state = load_file(str(weight), device='cpu')
F09:0062     model.load_state_dict(state, strict=True)
F09:0063     tensor_count = len(state)
F09:0064     del state
F09:0065     model = model.cuda().eval()
F09:0066     preprocessor = MODELS.build(cfg.data_preprocessor).cuda()
F09:0067     pipeline = Compose(cfg.test_pipeline)
F09:0068     meta = parse_pose_metainfo(dict(from_file=str(repo / 'sapiens/pose/configs/_base_/keypoints308.p
F09:0068+ y')))
F09:0069     names = [meta['keypoint_id2name'][i] for i in range(308)]
F09:0070     codec = UDPHeatmap(**{k: v for k, v in cfg.codec.items() if k != 'type'})
F09:0071     data = preprocessor(pipeline(dict(img=im, bbox=np.array([bbox], dtype=np.float32), bbox_score=np
F09:0071+ .ones(1, dtype=np.float32))))
F09:0072     inputs, sample = data['inputs'], data['data_samples']
F09:0073     torch.cuda.synchronize()
F09:0074     start = time.time()
F09:0075     with torch.inference_mode(), torch.autocast('cuda', dtype=torch.bfloat16):
F09:0076         heatmaps = model(inputs)
F09:0077         if cfg.val_cfg.get('flip_test', False):
F09:0078             flipped = model(inputs.flip(-1)).flip(-1)[: , meta['flip_indices']]

```

```

F09:0079         heatmaps = (heatmaps + flipped) / 2
F09:0080     heatmaps = heatmaps.float().cpu().numpy()
F09:0081     xy, scores = codec.decode(heatmaps[0])
F09:0082     xy = xy / sample['meta']['input_size'] * sample['meta']['bbox_scale'] + sample['meta']['bbox_cen
F09:0082+ ter'] - 0.5 * sample['meta']['bbox_scale']
F09:0083     xy, scores = xy[0], scores[0]
F09:0084     assert xy.shape == (308, 2) and np.isfinite(xy).all() and np.isfinite(scores).all()
F09:0085     joints = [dict(index=i, name=n, x=float(xy[i, 0]), y=float(xy[i, 1]), score=float(scores[i]), in
F09:0085+ side_image=bool(0 <= xy[i, 0] < w and 0 <= xy[i, 1] < h)) for i, n in enumerate(names)]
F09:0086     payload = dict(model=f'Sapiens2-{a.size.upper()}', image_path=str(a.image.resolve()),
F09:0087         image_sha256=hashlib.sha256(a.image.read_bytes()).hexdigest(),
F09:0088         image_width=w, image_height=h, joint_set='sociopticon308',
F09:0089         coordinate_space='original_image_pixels', bbox_xyxy=bbox,
F09:0090         visibility='unknown; heatmap scores are not calibrated visibility',
F09:0091         keypoints=joints, inference_seconds=time.time() - start)
F09:0092     (a.output_dir / 'keypoints.json').write_text(json.dumps(payload, indent=2) + '\n')
F09:0093     np.savez_compressed(a.output_dir / 'heatmaps.npz', heatmap=heatmaps[0])
F09:0094     body = im.copy()
F09:0095     for j in joints:
F09:0096         if j['name'] in {'nose', 'left_shoulder', 'right_shoulder', 'left_elbow', 'right_elbow', 'le
F09:0096+ ft_wrist', 'right_wrist', 'left_hip', 'right_hip', 'left_knee', 'right_knee', 'left_ankle', 'right_a
F09:0096+ nkle'} and j['inside_image']:
F09:0097             pt = (round(j['x']), round(j['y']))
F09:0098             color = (0, 220, 0) if j['score'] >= 0.3 else (0, 150, 255)
F09:0099             cv2.circle(body, pt, 5, color, -1, cv2.LINE_AA)
F09:0100             cv2.putText(body, f"{j['name']} {j['score']:.2f}", (pt[0] + 5, pt[1] - 6), cv2.FONT_HERSHEY
F09:0100+ SIMPLEX, .4, color, 1, cv2.LINE_AA)
F09:0101     cv2.imwrite(str(a.output_dir / 'body_keypoints.jpg'), body)
F09:0102     manifest = dict(checkpoint=str(weight), checkpoint_sha256=digest.hexdigest(),
F09:0103         configured_sha256_verified=True, strict_load=True, checkpoint_tensors=tensor_cou
F09:0103+ nt,
F09:0104         config=str(config), python=sys.executable, torch=torch.__version__,
F09:0105         input_size_hw=[1024, 768], precision='bf16',
F09:0106         flip_test=cfg.val_cfg.get('flip_test', False), manual_labels_used=False,
F09:0107         gpu=torch.cuda.get_device_name(), peak_gpu_memory_bytes=torch.cuda.max_memory_al
F09:0107+ located())
F09:0108     (a.output_dir / 'manifest.json').write_text(json.dumps(manifest, indent=2) + '\n')
F09:0109     print(json.dumps(payload, indent=2))
F09:0112 if __name__ == '__main__':
F09:0113     main()
F10:0001 from pathlib import Path
F10:0002 import sys, json, time, hashlib
F10:0003 import argparse
F10:0004 parser=argparse.ArgumentParser(description='Strict Sapiens2 29-class segmentation')
F10:0005 parser.add_argument('--repo-root', type=Path, required=True)
F10:0006 parser.add_argument('--dependencies', type=Path)
F10:0007 parser.add_argument('--checkpoint', type=Path, required=True)
F10:0008 parser.add_argument('--checkpoint-sha256', required=True)
F10:0009 parser.add_argument('--manifest', type=Path, required=True)
F10:0010 parser.add_argument('--output-dir', type=Path, required=True)

```

```

F10:0011 args=parser.parse_args()
F10:0012 REPO=args.repo_root;OUT=args.output_dir
F10:0013 OUT.mkdir(parents=True,exist_ok=False)
F10:0014 sys.path.insert(0,str(REPO))
F10:0015 if args.dependencies:sys.path.insert(0,str(args.dependencies))
F10:0016 import cv2,numpy as np,torch
F10:0017 import torch.nn.functional as F
F10:0018 from sapiens.engine.config import Config
F10:0019 from sapiens.engine.datasets import Compose
F10:0020 from sapiens.registry import MODELS
F10:0021 from sapiens.dense.visualizers import SegVisualizer
F10:0022 from sapiens.dense.datasets import DOME_CLASSES_29
F10:0023 from safetensors.torch import load_file
F10:0025 def write(p,x):p.write_text(json.dumps(x,indent=2,ensure_ascii=False))
F10:0026 def sha(p):
F10:0027     h=hashlib.sha256()
F10:0028     with p.open('rb') as f:
F10:0029         for b in iter(lambda:f.read(16*1024**2),b''):h.update(b)
F10:0030     return h.hexdigest()
F10:0032 def main():
F10:0033     cfgpath=REPO/' sapiens/dense/configs/seg/shutterstock_goliath/sapiens2_lb_seg_shutterstock_goliath-1
F10:0033+ 024x768.py';weight=args.checkpoint
F10:0034     digest=sha(weight);assert digest==args.checkpoint_sha256, 'Segmentation checkpoint SHA-256 mismatch
F10:0034+ '
F10:0035     cfg=Config.fromfile(cfgpath);cfg.model['backbone'].pop('init_cfg',None)
F10:0036     print('Building model',flush=True);model=MODELS.build(cfg.model);state=load_file(str(weight));model
F10:0036+ .load_state_dict(state,strict=True);count=len(state);del state
F10:0037     model=model.cuda().eval();prep=MODELS.build(cfg.data_preprocessor).cuda();pipe=Compose(cfg.test_pip
F10:0037+ eline)
F10:0038     viz=SegVisualizer(with_labels=False);palette=viz.class_palette[:29,:-1].copy()
F10:0039     write(OUT/'classes.json',{int(k):v for k,v in DOME_CLASSES_29.items()})
F10:0040     rows=json.loads(args.manifest.read_text())['frames'];records=[]
F10:0041     for r in rows:
F10:0042         t=time.time();d=OUT/'frames'/r['id'];d.mkdir(parents=True,exist_ok=True);im=cv2.imread(r['image'])
F10:0042+ ;assert im is not None
F10:0043         data=prep(pipe(dict(img=im)));inputs=data['inputs']
F10:0044         with torch.inference_mode(),torch.autocast('cuda',dtype=torch.bfloat16):logits=model(inputs)
F10:0045         logits=F.interpolate(logits.float(),size=im.shape[:2],mode='bilinear',align_corners=False);prob=lo
F10:0045+ gits.softmax(dim=1)[0].cpu().numpy();labels=prob.argmax(0).astype(np.uint8);confidence=prob.max(0)
F10:0046         assert prob.shape==(29,*im.shape[:2]) and np.isfinite(prob).all() and np.max(np.abs(prob.sum(0)-1)
F10:0046+ )<1e-5
F10:0047         np.save(d/'labels.npy',labels);cv2.imwrite(str(d/'labels.png'),labels);np.savez_compressed(d/'prob
F10:0047+ abilities.npz',probabilities=prob.astype(np.float16),class_ids=np.arange(29),coordinate_space=np.arr
F10:0047+ ay('original_image_pixels'))
F10:0048         cv2.imwrite(str(d/'original.jpg'),im);cv2.imwrite(str(d/'all_parts.jpg'),viz._visualize_segmentati
F10:0048+ on(im,labels));cv2.imwrite(str(d/'part_colors.png'),palette[labels])
F10:0049         focus=im.copy();region=np.isin(labels,[6,7,11,15,16,20]);focus[region]=(im[region]*.5+palette[1abe
F10:0049+ ls[region]]*.5).astype(np.uint8)
F10:0050         boundary=im.copy();stats={}
F10:0051         for k in [6,7,11,15,16,20]:

```

```

F10:0052     mask=(labels==k).astype(np.uint8);contours, _=cv2.findContours(mask,cv2.RETR_EXTERNAL,cv2.CHAIN_AP
F10:0052+ PROX_SIMPLE):color=tuple(int(x) for x in palette[k]);cv2.drawContours(boundary,contours,-1,color,2)
F10:0053     cv2.imwrite(str(d/f'mask_{k:02d}.png'),mask*255)
F10:0054     ys,xs=np.where(mask);stats[k]=dict(name=DOME_CLASSES_29[k]['name'],pixels=int(mask.sum()),mean_so
F10:0054+ ftmax=float(confidence[mask>0].mean()) if mask.any() else None,high_confidence_pixels=int(((mask>0)&
F10:0054+ (confidence>=.8)).sum()),bbox_xyxy=[int(xs.min()),int(ys.min()),int(xs.max()+1),int(ys.max()+1)] if
F10:0054+ len(xs) else None)
F10:0055     cv2.imwrite(str(d/'arms_hands.jpg'),focus);cv2.imwrite(str(d/'boundaries.jpg'),boundary);cv2.imwri
F10:0055+ te(str(d/'confidence.jpg'),cv2.applyColorMap((confidence*255).astype(np.uint8),cv2.COLORMAP_VIRIDIS)
F10:0055+ )
F10:0056     rec=dict(id=r['id'],camera=r.get('camera','unknown'),image=r['image'],image_sha256=sha(Path(r['ima
F10:0056+ ge'])),original_hw=list(im.shape[:2]),network_input_shape=list(inputs.shape),inference_seconds=time.
F10:0056+ time()-t,parts=stats,softmax_is_not_calibrated_accuracy=True)
F10:0057     write(d/'stats.json',rec);records.append(rec);print(' DONE',r['id'],round(time.time()-t,2),{x:stats
F10:0057+ [x]['pixels'] for x in stats},flush=True)
F10:0058     write(OUT/'manifest.json',dict(frames=records,checkpoint=str(weight),checkpoint_sha256=digest,confi
F10:0058+ gured_sha256_verified=True,strict_load=True,tensor_count=count,config=str(cfgpath),config_sha256=sha
F10:0058+ (cfgpath),python=sys.executable,torch=torch.__version__,precision='bfloat16 autocast; float32 interp
F10:0058+ olation/softmax',preprocessing='official test_pipeline: 1024x768 direct resize keep_ratio=False; BGR
F10:0058+ to RGB; official normalization',postprocessing='bilinear logits resize to original H,W align_corner
F10:0058+ s=False; softmax; argmax',manual_annotations_used=False,mesh_optimization_run=False))
F10:0059 if __name__=='__main__':main()
F11:0001 #!/usr/bin/env python3
F11:0002 """Run WiLoR hand detection/reconstruction on a configurable image manifest."""
F11:0004 import argparse
F11:0005 import json
F11:0006 from pathlib import Path
F11:0008 import cv2
F11:0009 import numpy as np
F11:0010 import torch
F11:0011 from scipy.spatial.transform import Rotation
F11:0013 from wilor.datasets.vitdet_dataset import ViTDetDataset
F11:0014 from wilor.models import load_wilor
F11:0015 from wilor.utils import recursive_to
F11:0016 from wilor.utils.camera import cam_crop_to_full
F11:0017 from wilor.utils.yolo_loader import load_yolo_detector
F11:0020 def parse_args():
F11:0021     ap = argparse.ArgumentParser(description=__doc__)
F11:0022     ap.add_argument("--manifest", type=Path, required=True)
F11:0023     ap.add_argument("--output-dir", type=Path, required=True)
F11:0024     ap.add_argument("--checkpoint", type=Path, required=True)
F11:0025     ap.add_argument("--config", type=Path, required=True)
F11:0026     ap.add_argument("--detector", type=Path, required=True)
F11:0027     ap.add_argument("--confidence", type=float, default=0.20)
F11:0028     ap.add_argument("--rescale-factor", type=float, default=2.0)
F11:0029     return ap.parse_args()
F11:0032 def project(points, translation, focal_length, image_size_wh):
F11:0033     width, height = map(float, image_size_wh)
F11:0034     xyz = points + translation[None]
F11:0035     uv = xyz[:, :2] / np.maximum(xyz[:, 2:3], 1e-6)

```

```

F11:0036     uv *= float(focal_length)
F11:0037     uv += np.asarray([width / 2.0, height / 2.0], np.float32)
F11:0038     return uv
F11:0041 def write_obj(path, vertices, faces):
F11:0042     with path.open("w", encoding="utf-8") as handle:
F11:0043         for vertex in vertices:
F11:0044             handle.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
F11:0045         for face in faces:
F11:0046             handle.write(f"f {face[0] + 1} {face[1] + 1} {face[2] + 1}\n")
F11:0049 def main():
F11:0050     args = parse_args()
F11:0051     if args.output_dir.exists():
F11:0052         raise FileExistsError(args.output_dir)
F11:0053     for name in ("npz", "json", "mesh", "overlay"):
F11:0054         (args.output_dir / name).mkdir(parents=True, exist_ok=True)
F11:0056     rows = json.loads(args.manifest.read_text())['frames']
F11:0057     images = [Path(r['image']) for r in rows]
F11:0058     if len({p.stem for p in images}) != len(images):
F11:0059         raise ValueError('WiLoR input stems must be unique; runner stages images by frame id')
F11:0061     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
F11:0062     model, model_cfg = load_wilor(str(args.checkpoint), str(args.config))
F11:0063     detector = load_yolo_detector(str(args.detector))
F11:0064     model = model.to(device).eval()
F11:0065     detector = detector.to(device)
F11:0066     faces = np.asarray(model.mano.faces, np.int32)
F11:0067     report = {"method": "WiLoR detector + MANO regression", "frames": []}
F11:0069     for image_path in images:
F11:0070         image = cv2.imread(str(image_path), cv2.IMREAD_COLOR)
F11:0071         result = detector(image, conf=args.confidence, verbose=False)[0]
F11:0072         boxes = result.bboxes.xyxy.detach().cpu().numpy().astype(np.float32)
F11:0073         scores = result.bboxes.conf.detach().cpu().numpy().astype(np.float32)
F11:0074         right = result.bboxes.cls.detach().cpu().numpy().astype(np.float32)
F11:0075         frame = {"image": str(image_path), "detections": []}
F11:0076         overlay = image.copy()
F11:0077         focal_value = float(model_cfg.EXTRA.FOCAL_LENGTH / model_cfg.MODEL.IMAGE_SIZE * max(image.sh
F11:0077+ ape[:2]))
F11:0078         if len(boxes) == 0:
F11:0079             np.savez_compressed(args.output_dir / 'npz' / f'{image_path.stem}_wilor.npz',
F11:0080                                 detector_confidence=np.empty(0, np.float32), is_right=np.empty(0, np.f
F11:0080+ loat32),
F11:0081                                 joints_2d=np.empty((0, 21, 2), np.float32), mano_faces=faces,
F11:0082                                 image_size_wh=np.array(image.shape[1:-1]), focal_length_px=np.array
F11:0082+ (focal_value),
F11:0083                                 image_sha256=np.array(__import__('hashlib').sha256(image_path.read_b
F11:0083+ ytes()).hexdigest()))
F11:0084         cv2.imwrite(str(args.output_dir / 'overlay' / f'{image_path.stem}_wilor.jpg'), overlay)
F11:0085         report['frames'].append(frame)
F11:0086         continue
F11:0088     dataset = ViTDetDataset(model_cfg, image, boxes, right, rescale_factor=args.rescale_factor)
F11:0089     loader = torch.utils.data.DataLoader(dataset, batch_size=16, shuffle=False, num_workers=0)

```



```

F11:0090         arrays = {key: [] for key in (
F11:0091             "bbox_xyxy", "detector_confidence", "is_right", "global_orient_aa",
F11:0092             "hand_pose_aa", "betas", "cam_t", "joints_3d_local", "joints_2d",
F11:0093             "vertices_3d_local",
F11:0094         )}
F11:0095         offset = 0
F11:0096         for batch in loader:
F11:0097             batch = recursive_to(batch, device)
F11:0098             with torch.no_grad():
F11:0099                 output = model(batch)
F11:0100                 pred_mano = output["pred_mano_params"]
F11:0101                 pred_cam = output["pred_cam"].clone()
F11:0102                 pred_cam[:, 1] *= 2 * batch["right"] - 1
F11:0103                 image_size = batch["img_size"].float()
F11:0104                 focal = model_cfg.EXTRA.FOCAL_LENGTH / model_cfg.MODEL.IMAGE_SIZE * image_size.max()
F11:0105                 translations = cam_crop_to_full(
F11:0106                     pred_cam, batch["box_center"].float(), batch["box_size"].float(),
F11:0107                     image_size, focal,
F11:0108                 ).detach().cpu().numpy()
F11:0109                 vertices_batch = output["pred_vertices"].detach().cpu().numpy()
F11:0110                 joints_batch = output["pred_keypoints_3d"].detach().cpu().numpy()
F11:0111                 hand_rot = pred_mano["hand_pose"].detach().cpu().numpy()
F11:0112                 global_rot = pred_mano["global_orient"].detach().cpu().numpy()
F11:0113                 betas = pred_mano["betas"].detach().cpu().numpy()
F11:0114                 right_batch = batch["right"].detach().cpu().numpy()
F11:0115                 size_batch = image_size.detach().cpu().numpy()
F11:0116                 for index in range(len(vertices_batch)):
F11:0117                     source_index = offset + index
F11:0118                     is_right = bool(right_batch[index] > 0.5)
F11:0119                     vertices = vertices_batch[index].copy()
F11:0120                     joints = joints_batch[index].copy()
F11:0121                     mirror = 1.0 if is_right else -1.0
F11:0122                     vertices[:, 0] *= mirror
F11:0123                     joints[:, 0] *= mirror
F11:0124                     uv = project(joints, translations[index], float(focal), size_batch[index])
F11:0125                     global_aa = Rotation.from_matrix(global_rot[index, 0]).as_rotvec().astype(np.float32)
F11:0125+ )
F11:0126                     hand_aa = Rotation.from_matrix(hand_rot[index]).as_rotvec().astype(np.float32)
F11:0127                     arrays["bbox_xyxy"].append(boxes[source_index])
F11:0128                     arrays["detector_confidence"].append(scores[source_index])
F11:0129                     arrays["is_right"].append(float(is_right))
F11:0130                     arrays["global_orient_aa"].append(global_aa)
F11:0131                     arrays["hand_pose_aa"].append(hand_aa)
F11:0132                     arrays["betas"].append(betas[index])
F11:0133                     arrays["cam_t"].append(translations[index])
F11:0134                     arrays["joints_3d_local"].append(joints)
F11:0135                     arrays["joints_2d"].append(uv)
F11:0136                     arrays["vertices_3d_local"].append(vertices)
F11:0137                     rec = {
F11:0138                         "index": source_index,

```

```

F11:0139         "is_right": is_right,
F11:0140         "confidence": float(scores[source_index]),
F11:0141         "bbox_xyxy": boxes[source_index].tolist(),
F11:0142         "joints_2d": uv.tolist(),
F11:0143         "global_orient_aa": global_aa.tolist(),
F11:0144         "hand_pose_aa": hand_aa.tolist(),
F11:0145         "betas": betas[index].tolist(),
F11:0146         "cam_t": translations[index].tolist(),
F11:0147     }
F11:0148     frame["detections"].append(rec)
F11:0149     color = (0, 220, 255) if is_right else (255, 180, 0)
F11:0150     p1 = tuple(np rint(boxes[source_index, :2]).astype(int))
F11:0151     p2 = tuple(np rint(boxes[source_index, 2:]).astype(int))
F11:0152     cv2.rectangle(overlay, p1, p2, color, 2)
F11:0153     cv2.putText(overlay, f"{'R' if is_right else 'L'} {scores[source_index]:.2f}", p1,
F11:0154                 cv2.FONT_HERSHEY_SIMPLEX, 0.65, color, 2, cv2.LINE_AA)
F11:0155     for point in uv:
F11:0156         if np.isfinite(point).all():
F11:0157             cv2.circle(overlay, tuple(np rint(point).astype(int)), 2, color, -1, cv2.LINE_AA)
F11:0157+ E_AA)
F11:0158         write_obj(
F11:0159             args.output_dir / "mesh" / f"{image_path.stem}_hand{source_index}_{'R' if is_right
F11:0159+ ht else 'L'}.obj",
F11:0160             vertices + translations[index][None], faces,
F11:0161         )
F11:0162         offset += len(vertices_batch)
F11:0163     np.savez_compressed(
F11:0164         args.output_dir / "npz" / f"{image_path.stem}_wilor.npz",
F11:0165         **{key: np.asarray(value) for key, value in arrays.items()},
F11:0166         mano_faces=faces,
F11:0167         image_size_wh=np.array(image.shape[1::-1]), focal_length_px=np.array(focal_value),
F11:0168         image_sha256=np.array(__import__('hashlib').sha256(image_path.read_bytes()).hexdigest())
F11:0169+ ,
F11:0170     )
F11:0171     (args.output_dir / "json" / f"{image_path.stem}_wilor.json").write_text(
F11:0172         json.dumps(frame, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
F11:0173     )
F11:0174     cv2.imwrite(str(args.output_dir / "overlay" / f"{image_path.stem}_wilor.jpg"), overlay)
F11:0175     report["frames"].append(frame)
F11:0176     print(image_path.name, len(frame["detections"]))
F11:0177     (args.output_dir / "wilor_report.json").write_text(
F11:0178         json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
F11:0179     )
F11:0180     )
F11:0183 if __name__ == "__main__":
F11:0184     main()
F12:0001 """Joint mappings, bounded pose increments, and rigid rotation math. No annotation reader."""
F12:0002 import numpy as np
F12:0003 import torch
F12:0005 JOINT_MAPPING = {
F12:0006     "nose": "Nose",

```

```

F12:0007     "left_shoulder": "L_Shoulder",
F12:0008     "right_shoulder": "R_Shoulder",
F12:0009     "left_elbow": "L_Elbow",
F12:0010     "right_elbow": "R_Elbow",
F12:0011     "left_wrist": "L_Wrist",
F12:0012     "right_wrist": "R_Wrist",
F12:0013     "left_hip": "L_Hip",
F12:0014     "right_hip": "R_Hip",
F12:0015     "left_knee": "L_Knee",
F12:0016     "right_knee": "R_Knee",
F12:0017     "left_ankle": "L_Ankle",
F12:0018     "right_ankle": "R_Ankle",
F12:0019 }
F12:0022 BODY_POSE_NAMES = (
F12:0023     "L_Hip", "R_Hip", "Spine_1", "L_Knee", "R_Knee", "Spine_2",
F12:0024     "L_Ankle", "R_Ankle", "Spine_3", "L_Foot", "R_Foot", "Neck",
F12:0025     "L_Collar", "R_Collar", "Head", "L_Shoulder", "R_Shoulder",
F12:0026     "L_Elbow", "R_Elbow", "L_Wrist", "R_Wrist",
F12:0027 )
F12:0030 MAX_DELTA_DEG = {
F12:0031     "L_Hip": 18.0, "R_Hip": 18.0,
F12:0032     "Spine_1": 10.0, "Spine_2": 10.0, "Spine_3": 12.0,
F12:0033     "L_Knee": 22.0, "R_Knee": 22.0,
F12:0034     "L_Ankle": 14.0, "R_Ankle": 14.0,
F12:0035     "Neck": 12.0, "Head": 12.0,
F12:0036     "L_Collar": 15.0, "R_Collar": 15.0,
F12:0037     "L_Shoulder": 28.0, "R_Shoulder": 28.0,
F12:0038     "L_Elbow": 25.0, "R_Elbow": 25.0,
F12:0039     "L_Wrist": 15.0, "R_Wrist": 15.0,
F12:0040 }
F12:0043 def save_obj(vertices, faces, path):
F12:0044     with path.open("w", encoding="utf-8") as output:
F12:0045         for vertex in vertices:
F12:0046             output.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
F12:0047         for face in faces:
F12:0048             output.write(f"f {face[0] + 1} {face[1] + 1} {face[2] + 1}\n")
F12:0050 def axis_angle_rotation(axis_angle):
F12:0051     x, y, z = axis_angle.unbind()
F12:0052     zero = torch.zeros_like(x)
F12:0053     skew = torch.stack((zero, -z, y, z, zero, -x, -y, x, zero)).reshape(3, 3)
F12:0054     theta = torch.linalg.norm(axis_angle)
F12:0055     first = torch.sinc(theta / np.pi)
F12:0056     second = 0.5 * torch.sinc(theta / (2.0 * np.pi)) ** 2
F12:0057     eye = torch.eye(3, dtype=axis_angle.dtype, device=axis_angle.device)
F12:0058     return eye + first * skew + second * (skew @ skew)
F13:0001 #!/usr/bin/env python3
F13:0002 """Fit only SMPL-X global orientation and translation to Sapiens2 2D points.
F13:0004 Reuses the existing calibrated-fisheye rigid-fit helpers. Only automatic Sapiens2 targets are accept
F13:0004+ ed.
F13:0005 The body rotates about the SMPL-X pelvis; saved parameters are regenerated and

```

```

F13:0006 checked against the optimized geometry before any result is delivered.
F13:0007 """
F13:0008 import argparse
F13:0009 import hashlib
F13:0010 import json
F13:0011 import os
F13:0012 from pathlib import Path
F13:0014 import cv2
F13:0015 import numpy as np
F13:0016 import torch
F13:0017 import torch.nn.functional as F
F13:0020 def main():
F13:0021     p = argparse.ArgumentParser(description=__doc__)
F13:0022     for name in ['repo-root', 'source-npz', 'calibration', 'keypoints', 'image', 'output-dir']:
F13:0023         p.add_argument('--' + name, type=Path, required=True)
F13:0024     p.add_argument('--score-threshold', type=float, default=0.3)
F13:0025     p.add_argument('--steps', type=int, default=1800)
F13:0026     p.add_argument('--max-orient-delta-deg', type=float, default=60.)
F13:0027     p.add_argument('--device', choices=['cpu', 'cuda'], default='cpu', help='Rigid fitting is small;
F13:0027+ CPU avoids old CUDA numerical differences')
F13:0028     a = p.parse_args()
F13:0029     if not 0 <= a.score_threshold <= 1 or a.steps < 1:
F13:0030         raise ValueError('Invalid score threshold or step count')
F13:0031     os.environ['EGOSMPLX_REPO_ROOT'] = str(a.repo_root.resolve())
F13:0032     from egosmplx_pipeline.bootstrap import setup
F13:0033     setup(a.repo_root)
F13:0034     from egosmplx_pipeline.body_common import axis_angle_rotation, save_obj, JOINT_MAPPING
F13:0035     from FishEyeCalibrated import FishEyeCameraCalibrated
F13:0036     from utils.human_models import smpl_x
F13:0037     from utils.vis import render_mesh_fisheye
F13:0039     image = cv2.imread(str(a.image))
F13:0040     if image is None:
F13:0041         raise ValueError('Unreadable image')
F13:0042     h, w = image.shape[:2]
F13:0043     payload = json.loads(a.keypoints.read_text())
F13:0044     if payload['coordinate_space'] != 'original_image_pixels':
F13:0045         raise ValueError('Sapiens2 targets must use original image pixels')
F13:0046     if [payload['image_width'], payload['image_height']] != [w, h]:
F13:0047         raise ValueError('Target dimensions differ from image')
F13:0048     if payload['image_sha256'] != hashlib.sha256(a.image.read_bytes()).hexdigest():
F13:0049         raise ValueError('Target image hash mismatch')
F13:0050     camera = FishEyeCameraCalibrated(str(a.calibration.resolve()))
F13:0051     if tuple(map(int, camera.img_size)) != (w, h):
F13:0052         raise ValueError('Calibration dimensions differ from image')
F13:0053     raw = {k: v.copy() for k, v in np.load(a.source_npz).items()}
F13:0054     fixed_keys = ['body_pose', 'betas', 'left_hand_pose', 'right_hand_pose', 'jaw_pose', 'leye_pose'
F13:0054+ , 'reye_pose', 'expression']
F13:0055     for key in fixed_keys + ['global_orient', 'transl', 'smplx_mesh_cam', 'smplx_body_joints_3d_loc
F13:0055+ a 1']:
F13:0056         if key not in raw or not np.isfinite(raw[key]).all():

```

```

F13:0057         raise ValueError('Missing or nonfinite source field: ' + key)
F13:0058     dev = torch.device(a.device)
F13:0059     # Rigid composition and parameter regeneration require full FP32 mantissas;
F13:0060     # TF32 matrix products can introduce millimeter-scale discrepancies.
F13:0061     torch.backends.cuda.matmul.allow_tf32 = False
F13:0062     torch.backends.cudnn.allow_tf32 = False
F13:0063     t = lambda x: torch.as_tensor(x, dtype=torch.float32, device=dev)
F13:0064     layer = smpl_x.layer['neutral'].to(dev).eval()
F13:0065     fixed = {key: t(raw[key]).reshape(1, -1) for key in fixed_keys}
F13:0066     cam0 = t(raw['transl']).reshape(3)
F13:0067     glob0 = t(raw['global_orient']).reshape(1, 3)
F13:0068     with torch.no_grad():
F13:0069         initial = layer(global_orient=glob0, **fixed)
F13:0070         local = initial.joints[0, smpl_x.joint_idx[:25]].detach()
F13:0071         vertices = initial.vertices[0].detach()
F13:0072         pivot = initial.joints[0, 0].detach()
F13:0073         mesh_error = float((vertices + cam0 - t(raw['smplx_mesh_cam'])).abs().max())
F13:0074         joints_error = float((local - t(raw['smplx_body_joints_3d_local'])).abs().max())
F13:0075         print('Source reconstruction errors (m):', mesh_error, joints_error, flush=True)
F13:0076         # Legacy CUDA inference and full-precision CPU LBS differ by submillimeter
F13:0077         # rounding. Audit this separately from the much tighter final round trip.
F13:0078         if max(mesh_error, joints_error) > 1e-3:
F13:0079             raise ValueError(f'Source parameter reconstruction mismatch: mesh={mesh_error}, joints={join
F13:0079+ ts_error} m')
F13:0081     # Selection is determined solely by model scores and in-image coordinates.
F13:0082     lookup = {j['name']: j for j in payload['keypoints']}
F13:0083     if len(lookup) != len(payload['keypoints']):
F13:0084         raise ValueError('Duplicate keypoint names')
F13:0085     selected, excluded = [], []
F13:0086     for name, model_name in JOINT_MAPPING.items():
F13:0087         j = lookup.get(name)
F13:0088         valid = j is not None and np.isfinite([j['x'], j['y'], j['score']]).all()
F13:0089         valid = valid and 0 <= j['x'] < w and 0 <= j['y'] < h and j['score'] >= a.score_threshold
F13:0090         if valid:
F13:0091             selected.append(dict(name=name, model_name=model_name, index=smpl_x.joints_name.index(mo
F13:0091+ del_name), x=j['x'], y=j['y'], score=j['score']))
F13:0092         else:
F13:0093             excluded.append(dict(name=name, prediction=j, reason='missing, nonfinite, outside image,
F13:0093+ or below score threshold'))
F13:0094     if len(selected) < 4:
F13:0095         raise ValueError(f'Only {len(selected)} usable automatic targets; need at least four')
F13:0096     target = t([j['x'], j['y']] for j in selected)
F13:0097     if np.linalg.svd(target.cpu().numpy() - target.cpu().numpy().mean(0), compute_uv=False)[1] < 10:
F13:0098         raise ValueError('Automatic targets have insufficient 2D spatial spread')
F13:0099     indices = torch.tensor([j['index'] for j in selected], device=dev)
F13:0100     weights = t([np.clip(j['score'], 0, 1)**2 for j in selected])
F13:0101     cam = cam0.clone().requires_grad_(True)
F13:0102     delta = torch.zeros(3, device=dev, requires_grad=True)
F13:0103     scale = t([w, h])
F13:0104     def geometry():

```

```

F13:0105         rotation = axis_angle_rotation(delta)
F13:0106         joints = (local - pivot) @ rotation.t() + pivot + cam
F13:0107         mesh = (vertices - pivot) @ rotation.t() + pivot + cam
F13:0108         pixels = camera.world2camera_pytorch(joints)
F13:0109         return joints, mesh, pixels
F13:0110     with torch.no_grad():
F13:0111         _, _, before = geometry()
F13:0112         before = before.clone()
F13:0113         opt = torch.optim.Adam([{'params': [cam], 'lr': .005}, {'params': [delta], 'lr': .002}])
F13:0114         best_loss, best_state = float('inf'), None
F13:0115         history = []
F13:0116         for step in range(a.steps + 1):
F13:0117             opt.zero_grad()
F13:0118             joints, mesh, pixels = geometry()
F13:0119             residual = (pixels[indices] - target) / scale
F13:0120             robust = F.smooth_l1_loss(residual, torch.zeros_like(residual), beta=.005, reduction='none')
F13:0120+ .mean(1)
F13:0121             reprojection = (robust * weights).sum() / weights.sum()
F13:0122             prior_t = (((cam - cam0) / t([.25, .25, .50]))**2).mean()
F13:0123             loss = reprojection + 1e-4 * prior_t + 2e-4 * delta.square().mean() + F.relu(.02 - mesh[:, 2
F13:0123+ ].min()).square()
F13:0124             if not torch.isfinite(loss):
F13:0125                 raise RuntimeError('Nonfinite optimization objective')
F13:0126             value = float(loss.detach())
F13:0127             if value < best_loss:
F13:0128                 best_loss, best_state = value, (cam.detach().clone(), delta.detach().clone())
F13:0129             if step % 100 == 0 or step == a.steps:
F13:0130                 history.append(dict(step=step, loss=value, reprojection=float(reprojection.detach())))
F13:0131             if step == a.steps:
F13:0132                 break
F13:0133             loss.backward()
F13:0134             if not torch.isfinite(cam.grad).all() or not torch.isfinite(delta.grad).all():
F13:0135                 raise RuntimeError('Nonfinite rigid-fit gradient')
F13:0136             opt.step()
F13:0137             with torch.no_grad():
F13:0138                 norm = delta.norm()
F13:0139                 limit = np.deg2rad(a.max_orient_delta_deg)
F13:0140                 if norm > limit:
F13:0141                     delta.mul_(limit / norm)
F13:0142                     cam[2].clamp_(-.05, 3.)
F13:0143             with torch.no_grad():
F13:0144                 cam.copy_(best_state[0])
F13:0145                 delta.copy_(best_state[1])
F13:0146             joints, mesh, after = geometry()
F13:0147             delta_rotation = axis_angle_rotation(delta).cpu().numpy()
F13:0148             new_rotation = delta_rotation @ cv2.Rodrigues(raw['global_orient']).astype(np.float64).reshape
F13:0148+ e(3)) [0]
F13:0149             new_global = cv2.Rodrigues(new_rotation.astype(np.float64)) [0].astype(np.float32).reshape(1,
F13:0149+ 3)
F13:0150             regenerated = layer(global_orient=t(new_global), **fixed)

```

```

F13:0151         rebuilt_mesh = regenerated.vertices[0] + cam
F13:0152         rebuilt_local = regenerated.joints[0, smpl_x.joint_idx[:25]]
F13:0153         rebuilt_pixels = camera.world2camera_pytorch(rebuilt_local + cam)
F13:0154         regen_mesh_error = float((rebuilt_mesh - mesh).abs().max())
F13:0155         regen_pixel_error = float((rebuilt_pixels - after).abs().max())
F13:0156         if regen_mesh_error > 1e-4 or regen_pixel_error > .05:
F13:0157             print('REGEN_DIAGNOSTIC', dict(
F13:0158                 difference_mean=(rebuilt_mesh - mesh).mean(0).cpu().tolist(),
F13:0159                 difference_std=(rebuilt_mesh - mesh).std(0).cpu().tolist(),
F13:0160                 pivot=pivot.cpu().tolist(), regenerated_pivot=regenerated.joints[0, 0].cpu().tolist(
F13:0160+ ),
F13:0161                 rotation_roundtrip_error=float(np.max(np.abs(cv2.Rodrigues(new_global.astype(np.floa
F13:0161+ t64).reshape(3))[0] - new_rotation))),
F13:0162                 rotation_orthogonality_error=float(np.max(np.abs(new_rotation.T @ new_rotation - np.
F13:0162+ eye(3)))),
F13:0163                 delta=delta.cpu().tolist(), global_new=new_global.tolist(), flush=True)
F13:0164             raise RuntimeError(f'Regenerated geometry mismatch: {regen_mesh_error} m, {regen_pixel_e
F13:0164+ rror} px')
F13:0165         cpu = lambda x: x.detach().cpu().numpy()
F13:0166         saved = {k: v.copy() for k, v in raw.items()}
F13:0167         saved.update(global_orient=new_global,
F13:0168                     transl=cpu(cam).reshape(1, 3), smplx_mesh_cam=cpu(rebuilt_mesh),
F13:0169                     smplx_body_joints_3d_local=cpu(rebuilt_local),
F13:0170                     smplx_body_joints_3d_render_cam=cpu(rebuilt_local + cam),
F13:0171                     smplx_body_joints_2d_render_cam=cpu(rebuilt_pixels),
F13:0172                     sapiens2_target_joint_pixels=cpu(target), sapiens2_selected_joint_indices=cpu(indic
F13:0172+ es),
F13:0173                     sapiens2_target_scores=np.array([j['score'] for j in selected], dtype=np.float32),
F13:0174                     global_orient_delta_rotation=delta_rotation)
F13:0175         full_joints = regenerated.joints[0, smpl_x.joint_idx]
F13:0176         saved['smplx_joint_cam_root_relative'] = cpu(full_joints - full_joints[:1])
F13:0177         if not all(np.array_equal(saved[k], raw[k]) for k in fixed_keys):
F13:0178             raise RuntimeError('A fixed parameter was changed')
F13:0179         def errors(pred, truth):
F13:0180             d = np.linalg.norm(pred - truth, axis=1)
F13:0181             return dict(rmse_px=float(np.sqrt(np.mean(d**2))), mean_px=float(d.mean()), median_px=float(
F13:0181+ np.median(d)))
F13:0182         report = dict(source_npz=str(a.source_npz.resolve()), image=str(a.image.resolve()), calibration=
F13:0182+ str(a.calibration.resolve()),
F13:0183                     sapiens2_keypoints=str(a.keypoints.resolve()), optimized_parameters=['global_orien
F13:0183+ t', 'cam_trans'],
F13:0184                     fixed_parameters=fixed_keys, fixed_parameters_bitwise_equal=True, network_was_reru
F13:0184+ n=False,
F13:0185                     manual_labels_used_for_optimization=False, score_threshold=a.score_threshold,
F13:0186                     score_weight='clip(score,0,1)^2; scores are not calibrated visibility probabilitie
F13:0186+ s',
F13:0187                     selected=selected, excluded=excluded, initial_cam_trans_m=cpu(cam0).tolist(),
F13:0188                     optimized_cam_trans_m=cpu(cam).tolist(), initial_global_orient=raw['global_orient'
F13:0188+ ].tolist(),
F13:0189                     optimized_global_orient=new_global.tolist(), global_orient_delta_deg=float(np.rad2

```

```

F13:0189+ deg(np.linalg.norm(cpu(delta))),
F13:0190             source_reconstruction_max_abs_m=dict(mesh=mesh_error, joints=joints_error),
F13:0191             source_reconstruction_tolerance_m=1e-3, optimization_device=str(dev),
F13:0192             saved_parameter_regeneration_max_abs_m=regen_mesh_error,
F13:0193             saved_parameter_regeneration_max_abs_px=regen_pixel_error,
F13:0194             source_npz_sha256=hashlib.sha256(a.source_npz.read_bytes()).hexdigest(),
F13:0195             raw_to_sapiens2=errors(cpu(before[indices]), cpu(target)),
F13:0196             optimized_to_sapiens2=errors(cpu(rebuilt_pixels[indices]), cpu(target)), history=h
F13:0196+ istory,
F13:0197             minimum_vertex_z_m=float(rebuilt_mesh[:, 2].min()),
F13:0198             geometry_convention='SMPL-X vertices/joints include global orientation; add transl
F13:0198+ ; rotate about SMPL-X pelvis',
F13:0199             preserved_network_fields='Fields containing network or model_output and positionne
F13:0199+ t are original inference diagnostics')
F13:0200     a.output_dir.mkdir(parents=True, exist_ok=False)
F13:0201     np.savez_compressed(a.output_dir / 'optimized_params.npz', **saved)
F13:0202     save_obj(cpu(rebuilt_mesh), smpl_x.face, a.output_dir / 'optimized_mesh.obj')
F13:0203     overlays = []
F13:0204     for name, verts in [('raw', raw['smplx_mesh_cam']), ('optimized', cpu(rebuilt_mesh))]:
F13:0205         overlay = render_mesh_fisheye(image, verts, smpl_x.face, {}, camera, mode='wireframe', mesh_
F13:0205+ color=(255, 255, 255), alpha=.78, proj_trans=None)
F13:0206         cv2.imwrite(str(a.output_dir / f'{name}_wireframe.jpg'), overlay)
F13:0207         panel = cv2.resize(overlay, (640, round(h * 640 / w)))
F13:0208         cv2.rectangle(panel, (0, 0), (640, 38), (25, 25, 25), -1)
F13:0209         cv2.putText(panel, 'Raw EgoSMPLX' if name == 'raw' else 'Sapiens2 -> global orient + transla
F13:0209+ tion', (12, 25), cv2.FONT_HERSHEY_SIMPLEX, .62, (255, 255, 255), 1, cv2.LINE_AA)
F13:0210         overlays.append(panel)
F13:0211         cv2.imwrite(str(a.output_dir / 'comparison.jpg'), np.concatenate(overlays, axis=1))
F13:0212         diagnostic = image.copy()
F13:0213         for j, initial_xy, final_xy in zip(selected, cpu(before[indices]), cpu(rebuilt_pixels[indices]))
F13:0213+ :
F13:0214             target_xy = (round(j['x']), round(j['y']))
F13:0215             cv2.line(diagnostic, target_xy, tuple(np rint(final_xy).astype(int)), (0, 220, 220), 1, cv2.
F13:0215+ LINE_AA)
F13:0216             for xy, color in [(initial_xy, (255, 100, 0)), (final_xy, (0, 0, 255)), (target_xy, (0, 220,
F13:0216+ 0))]:
F13:0217                 cv2.circle(diagnostic, tuple(np rint(xy).astype(int)), 5, color, -1, cv2.LINE_AA)
F13:0218                 cv2.putText(diagnostic, j['name'], (target_xy[0] + 7, target_xy[1] - 7), cv2.FONT_HERSHEY_SI
F13:0218+ MPLEX, .4, (0, 220, 0), 1, cv2.LINE_AA)
F13:0219         cv2.putText(diagnostic, 'BLUE raw | GREEN Sapiens2 | RED optimized', (15, 30), cv2.FONT_HERSHEY_
F13:0219+ SIMPLEX, .65, (255, 255, 255), 2, cv2.LINE_AA)
F13:0220         cv2.imwrite(str(a.output_dir / 'keypoint_diagnostic.jpg'), diagnostic)
F13:0221         (a.output_dir / 'optimization_report.json').write_text(json.dumps(report, indent=2) + '\n')
F13:0222         print(json.dumps(report, indent=2))
F13:0225 if __name__ == '__main__':
F13:0226     main()
F14:0001 #!/usr/bin/env python3
F14:0002 """Optimize 19 body joints and small rigid changes using automatic points and visible contours.
F14:0004 Shape, hands, feet and facial parameters stay fixed. No local surface deformation.
F14:0005 """

```



```

F14:0006 import argparse
F14:0007 import hashlib
F14:0008 import json
F14:0009 import os
F14:0010 from pathlib import Path
F14:0011 import time
F14:0013 import cv2
F14:0014 import numpy as np
F14:0015 import torch
F14:0016 import torch.nn.functional as F
F14:0019 def main():
F14:0020     p = argparse.ArgumentParser(description=__doc__)
F14:0021     for key in ['repo-root', 'source-npz', 'image', 'calibration', 'automatic-keypoints', 'output-di
F14:0021+ r']:
F14:0022         p.add_argument('--' + key, type=Path, required=True)
F14:0023         p.add_argument('--steps', type=int, default=1600)
F14:0024         p.add_argument('--segmentation', type=Path, required=True)
F14:0025         p.add_argument('--silhouette-weight', type=float, default=.08)
F14:0026         a = p.parse_args()
F14:0027         os.environ['EGOSMPLX_REPO_ROOT'] = str(a.repo_root.resolve())
F14:0028         from egosmplx_pipeline.bootstrap import setup
F14:0029         setup(a.repo_root)
F14:0030         from egosmplx_pipeline.body_common import BODY_POSE_NAMES, MAX_DELTA_DEG, JOINT_MAPPING, save_ob
F14:0030+ j
F14:0031         from egosmplx_pipeline.contour import build_targets, contour_loss
F14:0032         from FishEyeCalibrated import FishEyeCameraCalibrated
F14:0033         from utils.human_models import smpl_x
F14:0034         from utils.vis import render_mesh_fisheye
F14:0036         a.output_dir.mkdir(parents=True, exist_ok=False)
F14:0037         image = cv2.imread(str(a.image))
F14:0038         if image is None:
F14:0039             raise ValueError('Unreadable image')
F14:0040         h, w = image.shape[:2]
F14:0041         camera = FishEyeCameraCalibrated(str(a.calibration))
F14:0042         assert tuple(map(int, camera.img_size)) == (w, h)
F14:0043         raw = {k: v.copy() for k, v in np.load(a.source_npz).items()}
F14:0044         auto = json.loads(a.automatic_keypoints.read_text())
F14:0045         assert auto['coordinate_space'] == 'original_image_pixels'
F14:0046         assert [auto['image_width'], auto['image_height']] == [w, h]
F14:0047         assert auto['image_sha256'] == hashlib.sha256(a.image.read_bytes()).hexdigest()
F14:0048         lookup = {j['name']: j for j in auto['keypoints']}
F14:0049         # Filter automatic observations before optimization.
F14:0050         names = [n for n in JOINT_MAPPING if n in lookup and np.isfinite([lookup[n][k] for k in ['x', 'y
F14:0050+ ', 'score']]).all()
F14:0051             and lookup[n]['score'] >= .3 and 0 <= lookup[n]['x'] < w and 0 <= lookup[n]['y'] < h]
F14:0052         if len(names) < 4:
F14:0053             raise ValueError('Too few usable automatic targets')
F14:0054         t = lambda x: torch.as_tensor(x, dtype=torch.float32)
F14:0055         indices = torch.tensor([smpl_x.joints_name.index(JOINT_MAPPING[n]) for n in names])
F14:0056         auto_xy = np.array([[lookup[n]['x'], lookup[n]['y']] for n in names], dtype=np.float32)

```

```

F14:0057     layer = smpl_x.layer['neutral'].cpu().eval()
F14:0058     layer.requires_grad_(False)
F14:0059     fixed_keys = ['betas', 'left_hand_pose', 'right_hand_pose', 'jaw_pose', 'leye_pose', 'reye_pose'
F14:0059+ , 'expression']
F14:0060     fixed = {k: t(raw[k]).reshape(1, -1) for k in fixed_keys}
F14:0061     body0, glob0, cam0 = t(raw['body_pose']).reshape(21, 3), t(raw['global_orient']).reshape(3), t(r
F14:0061+ aw['transl']).reshape(3)
F14:0062     active = list(MAX_DELTA_DEG)
F14:0063     active_indices = torch.tensor([BODY_POSE_NAMES.index(n) for n in active])
F14:0064     caps = t([np.deg2rad(MAX_DELTA_DEG[n]) for n in active])
F14:0065     wrists = [smpl_x.joints_name.index('L_Wrist'), smpl_x.joints_name.index('R_Wrist')]
F14:0066     def forward(body, glob, cam):
F14:0067         out = layer(body_pose=body.reshape(1, -1), global_orient=glob.reshape(1, 3), **fixed)
F14:0068         local = out.joints[0, smpl_x.joint_idx]
F14:0069         mesh = out.vertices[0] + cam
F14:0070         joints = local + cam
F14:0071         pixels = camera.world2camera_pytorch(joints)
F14:0072         return mesh, local, joints, pixels
F14:0073     with torch.no_grad():
F14:0074         initial = tuple(v.clone() for v in forward(body0, glob0, cam0))
F14:0075         initial_wrists = initial[2][wrists].norm(dim=1)
F14:0076         wrist_floor = initial_wrists * .85
F14:0077         source_error = float((initial[0] - t(raw['smplx_mesh_cam'])).abs().max())
F14:0078         assert source_error < 1e-5
F14:0079     def cpu(x):
F14:0080         return x.detach().numpy()
F14:0081     def error(pred, target):
F14:0082         d = np.linalg.norm(pred - target, axis=1)
F14:0083         return dict(rmse_px=float(np.sqrt(np.mean(d*d))), mean_px=float(d.mean()), median_px=float(n
F14:0083+ p.median(d)), per_joint_px={n: float(v) for n, v in zip(names, d)})
F14:0084     def angle_between(x, y):
F14:0085         rx, ry = cv2.Rodrigues(np.asarray(x, dtype=np.float64))[0], cv2.Rodrigues(np.asarray(y, dtyp
F14:0085+ e=np.float64))[0]
F14:0086         return float(np.rad2deg(np.arccos(np.clip((np.trace(rx @ ry.T)-1)/2, -1, 1))))
F14:0087     contour_data, contour_report = build_targets(a.segmentation, lookup, cpu(initial[0]), cpu(initia
F14:0087+ 1[2]), smpl_x, w, h)
F14:0088     (a.output_dir / 'contour_targets.json').write_text(json.dumps(contour_report, indent=2))
F14:0089     settings = dict(silhouette_weight=a.silhouette_weight, steps=a.steps, device='cpu', target_weigh
F14:0089+ ting='uniform automatic targets',
F14:0090                     body_lr=.0015, camera_lr=.0008, orient_lr=.0005,
F14:0091                     pose_prior_weight=.0008, camera_prior_weight=.0002, orient_prior_weight=.0004,
F14:0092                     smooth_l1_beta=.003, max_global_axis_angle_increment_deg=8.,
F14:0093                     body_axis_angle_increment_limits_deg=MAX_DELTA_DEG,
F14:0094                     wrist_distance_floor_fraction=.85, wrist_guard_weight=.002,
F14:0095                     wrist_distance_floor_m=cpu(wrist_floor).tolist(), minimum_vertex_z_m=.02,
F14:0096                     depth_barrier_onset_m=.03, depth_barrier_scale_m=.02, depth_barrier_weight=.02,
F14:0097                     best_candidate_rule='lowest total objective among candidates satisfying depth gu
F14:0097+ ards: manual evaluation never used')
F14:0099     def fit(label, target_xy):
F14:0100         start = time.time()

```

```

F14:0101         target = t(target_xy)
F14:0102         delta = torch.zeros((len(active), 3), requires_grad=True)
F14:0103         gd = torch.zeros(3, requires_grad=True)
F14:0104         cam = cam0.clone().requires_grad_(True)
F14:0105         opt = torch.optim.Adam([{'params': [delta], 'lr': .0015}, {'params': [gd], 'lr': .0005}, {'p
F14:0105+ arams': [cam], 'lr': .0008}])
F14:0106         best_loss, best = float('inf'), None
F14:0107         history = []
F14:0108         for step in range(a.steps + 1):
F14:0109             opt.zero_grad()
F14:0110             body = body0.clone()
F14:0111             body[active_indices] = body[active_indices] + delta
F14:0112             mesh, local, joints, pixels = forward(body, glob0 + gd, cam)
F14:0113             rep = F.smooth_l1_loss((pixels[indices] - target) / t([w, h]), torch.zeros_like(target),
F14:0113+ beta=.003)
F14:0114             pp = (delta / caps[:, None]).square().mean()
F14:0115             cp = ((cam - cam0) / t([.10, .10, .18])).square().mean()
F14:0116             gp = (gd / np.deg2rad(8.)).square().mean()
F14:0117             wrist_dist = joints[wrists].norm(dim=1)
F14:0118             guard = F.relu((wrist_floor - wrist_dist) / .05).square().mean()
F14:0119             # Start the barrier before the hard near plane. The legacy penalty
F14:0120             # in raw meters was too weak relative to normalized pixel residuals.
F14:0121             depth = F.relu((.03 - mesh[:, 2].min()) / .02).square()
F14:0122             sil = contour_loss(mesh, camera, contour_data, w, h)
F14:0123             loss = rep + .0008*pp + .0002*cp + .0004*gp + .002*guard + .02*depth + a.silhouette_weig
F14:0123+ ht*sil
F14:0124             if not torch.isfinite(loss):
F14:0125                 raise RuntimeError('Nonfinite objective')
F14:0126             feasible = bool((wrist_dist >= wrist_floor - 1e-6).all() and mesh[:, 2].min() >= .02 - 1
F14:0126+ e-6)
F14:0127             value = float(loss.detach())
F14:0128             if feasible and value < best_loss:
F14:0129                 best_loss = value
F14:0130                 best = (delta.detach().clone(), gd.detach().clone(), cam.detach().clone(), step)
F14:0131             if step % 100 == 0 or step == a.steps:
F14:0132                 item = dict(step=step, loss=value, reprojection=float(rep.detach()), feasible=feasib
F14:0132+ 1e,
F14:0133                             minimum_z_m=float(mesh[:, 2].min().detach()))
F14:0134                 history.append(item)
F14:0135                 print(label, item, flush=True)
F14:0136             if step == a.steps:
F14:0137                 break
F14:0138             loss.backward()
F14:0139             if not all(torch.isfinite(x.grad).all() for x in [delta, gd, cam]):
F14:0140                 raise RuntimeError('Nonfinite optimization gradients')
F14:0141             opt.step()
F14:0142             with torch.no_grad():
F14:0143                 delta.mul_(torch.minimum(torch.ones_like(caps), caps / delta.norm(dim=1).clamp_min(1
F14:0143+ e-8))[:, None])
F14:0144                 gd.mul_(torch.clamp(t(np.deg2rad(8.)) / gd.norm(), clamp_min(1e-8), max=1.))

```

```

F14:0145         cam[2].clamp_(.05, 3.)
F14:0146         if best is None:
F14:0147             raise RuntimeError('No feasible refinement candidate')
F14:0148         with torch.no_grad():
F14:0149             bd, gg, cc, best_step = best
F14:0150             final_body = body0.clone()
F14:0151             final_body[active_indices] += bd
F14:0152             final_global = glob0 + gg
F14:0153             mesh, local, joints, pixels = forward(final_body, final_global, cc)
F14:0154             dst = a.output_dir / label
F14:0155             dst.mkdir()
F14:0156             saved = {k: v.copy() for k, v in raw.items()}
F14:0157             saved.update(body_pose_rigid_input=raw['body_pose'], global_orient_rigid_input=raw['global_o
F14:0157+ rient'], transl_rigid_input=raw['transl'],
F14:0158                 body_pose=cpu(final_body).reshape(raw['body_pose'].shape), global_orient=cpu(fi
F14:0158+ nal_global).reshape(1, 3), transl=cpu(cc).reshape(1, 3),
F14:0159                 smplx_mesh_cam=cpu(mesh), smplx_body_joints_3d_local=cpu(local[:25]), smplx_bod
F14:0159+ y_joints_3d_render_cam=cpu(joints[:25]),
F14:0160                 smplx_body_joints_2d_render_cam=cpu(pixels[:25]), smplx_joint_cam_root_relative
F14:0160+ =cpu(local-local[:1]),
F14:0161                 stage2_target_joint_pixels=target_xy, stage2_selected_joint_indices=cpu(indices
F14:0161+ ))
F14:0162             assert all(np.array_equal(saved[k], raw[k]) for k in fixed_keys)
F14:0163             np.savez_compressed(dst / 'refined_params.npz', **saved)
F14:0164             # Independently reload the actual artifact and regenerate its geometry.
F14:0165             reloaded = np.load(dst / 'refined_params.npz')
F14:0166             assert all(np.isfinite(reloaded[k]).all() for k in reloaded.files if np.issubdtype(reloaded[
F14:0166+ k].dtype, np.number))
F14:0167             with torch.no_grad():
F14:0168                 rm, _, _, rp = forward(t(reloaded['body_pose']).reshape(21, 3), t(reloaded['global_orien
F14:0168+ t']).reshape(3), t(reloaded['transl']).reshape(3))
F14:0169                 reload_mesh_error = float(np.abs(cpu(rm)-reloaded['smplx_mesh_cam']).max())
F14:0170                 reload_pixel_error = float(np.abs(cpu(rp[:25])-reloaded['smplx_body_joints_2d_render_cam'])).
F14:0170+ max())
F14:0171             assert reload_mesh_error < 1e-5 and reload_pixel_error < .01
F14:0172             save_obj(cpu(mesh), smplx_x.face, dst / 'refined_mesh.obj')
F14:0173             overlay = render_mesh_fisheye(image, cpu(mesh), smplx_x.face, {}, camera, mode='wireframe', m
F14:0173+ esh_color=(255, 255, 255), alpha=.78, proj_trans=None)
F14:0174             cv2.imwrite(str(dst / 'refined_wireframe.jpg'), overlay)
F14:0175             diag = image.copy()
F14:0176             for n, target_pt, pred in zip(names, target_xy, cpu(pixels[indices])):
F14:0177                 tx, px = tuple(np rint(target_pt).astype(int)), tuple(np rint(pred).astype(int))
F14:0178                 cv2.line(diag, tx, px, (0, 220, 220), 1, cv2.LINE_AA)
F14:0179                 cv2.circle(diag, tx, 6, (0, 220, 0), -1, cv2.LINE_AA)
F14:0180                 cv2.circle(diag, px, 4, (0, 0, 255), -1, cv2.LINE_AA)
F14:0181                 cv2.putText(diag, n, (tx[0]+6, tx[1]-6), cv2.FONT_HERSHEY_SIMPLEX, .4, (0, 180, 0), 1, c
F14:0181+ v2.LINE_AA)
F14:0182             cv2.imwrite(str(dst / 'keypoint_diagnostic.jpg'), diag)
F14:0183             increments = np.rad2deg(cpu(bd.norm(dim=1)))
F14:0184             actual_angles = {n: angle_between(cpu(final_body[i]), cpu(body0[i])) for n, i in zip(active,

```

```

F14:0184+ active_indices)}}
F14:0185         report = dict(branch=label, target_source='Sapiens2',
F14:0186                 manual_labels_used_for_optimization=False,
F14:0187                 target_fit_before=error(cpu(initial[3][indices]), target_xy), target_fit_after
F14:0187+ =error(cpu(pixels[indices]), target_xy),
F14:0188                 best_step=best_step, best_objective=best_loss, elapsed_seconds=time.time()-sta
F14:0188+ rt,
F14:0189                 pose_increment_deg={n: float(v) for n, v in zip(active, increments)}, pose_geo
F14:0189+ desic_change_deg=actual_angles,
F14:0190                 joints_at_increment_limit=[n for n, v in zip(active, increments) if v >= MAX_D
F14:0190+ ELTA_DEG[n]-.05],
F14:0191                 global_geodesic_change_deg=angle_between(cpu(final_global), cpu(glob0)),
F14:0192                 cam_trans_m=cpu(cc).tolist(), cam_trans_change_m=cpu(cc-cam0).tolist(),
F14:0193                 wrist_camera_distance_m=cpu(joints[wrists].norm(dim=1)).tolist(),
F14:0194                 initial_wrist_camera_distance_m=cpu(initial_wrists).tolist(), minimum_vertex_z
F14:0194+ _m=float(mesh[:, 2].min()),
F14:0195                 fixed_fields_bitwise_equal=fixed_keys, reload_mesh_error_m=reload_mesh_error,
F14:0195+ reload_pixel_error_px=reload_pixel_error,
F14:0196                 history=history)
F14:0197         (dst / 'refinement_report.json').write_text(json.dumps(report, indent=2)+'\n')
F14:0198         return report, cpu(pixels[indices]), cpu(mesh), overlay, cpu(final_body)
F14:0200     automatic = fit('automatic', auto_xy)
F14:0201     if True:
F14:0202         rows = {
F14:0203             'rigid_shared_input': {'against_sapiens2': error(cpu(initial[3][indices]), auto_xy)},
F14:0204             'automatic_stage2': {'against_sapiens2': error(automatic[1], auto_xy)},
F14:0205         }
F14:0206         summary = dict(source_npz=str(a.source_npz), source_sha256=hashlib.sha256(a.source_npz.read_
F14:0206+ bytes()).hexdigest(),
F14:0207                 automatic_keypoints=str(a.automatic_keypoints), image=str(a.image), calibrati
F14:0207+ on=str(a.calibration),
F14:0208                 comparison='Automatic Sapiens2 stage-two refinement from the supplied rigid i
F14:0208+ nitialization',
F14:0209                 names=names, settings=settings, source_mesh_reload_error_m=source_error, resu
F14:0209+ lts=rows,
F14:0210                 automatic_diagnostics=automatic[0])
F14:0211         (a.output_dir / 'comparison_summary.json').write_text(json.dumps(summary, indent=2)+'\n')
F14:0212         rigid_overlay = render_mesh_fisheye(image, cpu(initial[0]), smpl_x.face, {}, camera, mode='w
F14:0212+ ireframe', mesh_color=(255, 255, 255), alpha=.78, proj_trans=None)
F14:0213         panels = []
F14:0214         for title, overlay, key in [('Rigid input', rigid_overlay, 'rigid_shared_input'), ('Stage 2:
F14:0214+ Sapiens2 targets', automatic[3], 'automatic_stage2')]:
F14:0215             panel = cv2.resize(overlay, (640, round(h*640/w)))
F14:0216             cv2.rectangle(panel, (0, 0), (640, 62), (25, 25, 25), -1)
F14:0217             cv2.putText(panel, title, (12, 25), cv2.FONT_HERSHEY_SIMPLEX, .65, (255, 255, 255), 1, c
F14:0217+ v2.LINE_AA)
F14:0218             cv2.putText(panel, f"Sapiens2 RMSE: {rows[key]['against_sapiens2']['rmse_px']:.2f} px",
F14:0218+ (12, 49), cv2.FONT_HERSHEY_SIMPLEX, .55, (210, 210, 210), 1, cv2.LINE_AA)
F14:0219             panels.append(panel)
F14:0220             cv2.imwrite(str(a.output_dir / 'stage2_comparison.jpg'), np.concatenate(panels, axis=1))

```

```

F14:0221         print('AUTOMATIC_ONLY', json.dumps(rows, indent=2), flush=True)
F14:0222         return
F14:0225 if __name__ == '__main__':
F14:0226     main()
F15:0001 """Visible forearm silhouette constraints on SMPL-X pose parameters only.
F15:0003 Side identity comes from pose landmarks. Segmentation left/right classes are
F15:0004 merged. Hidden boundaries and short/ambiguous cross-sections are not targets.
F15:0005 """
F15:0006 from pathlib import Path
F15:0007 import cv2
F15:0008 import numpy as np
F15:0009 import torch
F15:0010 import torch.nn.functional as F
F15:0012 ARM_CLASSES = [6, 7, 11, 15, 16, 20]
F15:0015 def build_targets(directory, lookup, vertices, joints, model, width, height):
F15:0016     directory = Path(directory)
F15:0017     labels = np.load(directory / 'labels.npy', allow_pickle=False)
F15:0018     with np.load(directory / 'probabilities.npz', allow_pickle=False) as z:
F15:0019         prob = z['probabilities'].astype(np.float32)
F15:0020     if labels.shape != (height, width) or prob.shape != (29, height, width):
F15:0021         raise ValueError('Segmentation dimensions differ from image')
F15:0022     if not np.isfinite(prob).all():
F15:0023         raise ValueError('Nonfinite segmentation')
F15:0024     skin = np.isin(labels, ARM_CLASSES)
F15:0025     confidence = prob[ARM_CLASSES].sum(0)
F15:0026     weights = model.layer['neutral'].lbs_weights.detach().cpu().numpy()
F15:0027     hand_ids = np.concatenate(list(model.hand_vertex_idx.values()))
F15:0028     outside = cv2.distanceTransform((~skin).astype(np.uint8), cv2.DIST_L2, 5)
F15:0029     data, report = [], {}
F15:0030     for si, side in enumerate(['left', 'right']):
F15:0031         elbow = lookup.get(side + '_elbow')
F15:0032         wrist = lookup.get(side + '_wrist')
F15:0033         observed = all(q is not None and q['score'] >= .3 and
F15:0034                        np.isfinite([q['x'], q['y'], q['score']]).all() and
F15:0035                        0 <= q['x'] < width and 0 <= q['y'] < height
F15:0036                        for q in [elbow, wrist])
F15:0037         if not observed:
F15:0038             report[side] = dict(enabled=False, reason='unreliable_automatic_landmarks')
F15:0039             continue
F15:0040         origin = np.array([elbow['x'], elbow['y']])
F15:0041         vector = np.array([wrist['x'], wrist['y']]) - origin
F15:0042         length = np.linalg.norm(vector)
F15:0043         if length <= 35:
F15:0044             report[side] = dict(enabled=False, reason='forearm_too_short_in_image')
F15:0045             continue
F15:0046         normal = np.array([-vector[1], vector[0]]) / length
F15:0047         half = min(70, max(25, length * .30))
F15:0048         bins = []
F15:0049         for position in np.linspace(.15, .88, 12):
F15:0050             offsets = np.arange(-half, half + 1)

```

```

F24:0944     iterations,
F24:0945     learning_rate,
F24:0946 ):
F24:0947     """Fit tx, ty, and positive tz while keeping pose and rotation fixed."""
F24:0948     if iterations <= 0:
F24:0949         raise ValueError("--camera-fit-iters must be positive")
F24:0950     if learning_rate <= 0:
F24:0951         raise ValueError("--camera-fit-lr must be positive")
F24:0952     device = body_joints_local.device
F24:0953     dtype = body_joints_local.dtype
F24:0954     indices = torch.as_tensor(CAMERA_FIT_BODY_INDICES, device=device, dtype=torch.long)
F24:0955     joints = body_joints_local.index_select(0, indices).detach()
F24:0956     targets = torch.as_tensor(
F24:0957         body_targets_px[CAMERA_FIT_BODY_INDICES], device=device, dtype=dtype
F24:0958     )
F24:0959     initial = initial_translation.detach().to(device=device, dtype=dtype)
F24:0960     identity = torch.eye(3, device=device, dtype=dtype)
F24:0961     def project(translation):
F24:0962         return fisheye_cam.world2camera_pytorch(joints + translation[None, :])
F24:0963     with torch.no_grad():
F24:0964         initial_projection = project(initial)
F24:0965         initial_error = torch.linalg.vector_norm(
F24:0966             initial_projection - targets, dim=1
F24:0967         ).mean()
F24:0968     initial_z = max(0.1, float(initial[2].item()))
F24:0969     depth_starts = sorted(set([initial_z, 0.5, 0.75, 1.0, 1.5, 2.0]))
F24:0970     best = None
F24:0971     for depth_start in depth_starts:
F24:0972         parameter = torch.tensor(
F24:0973             [
F24:0974                 float(initial[0]),
F24:0975                 float(initial[1]),
F24:0976                 float(np.log(depth_start)),
F24:0977             ],
F24:0978             device=device,
F24:0979             dtype=dtype,
F24:0980             requires_grad=True,
F24:0981         )
F24:0982         optimizer = torch.optim.Adam([parameter], lr=learning_rate)
F24:0983         for _ in range(iterations):
F24:0984             translation = torch.stack(
F24:0985                 (parameter[0], parameter[1], torch.exp(parameter[2]))
F24:0986             )
F24:0987             projection = project(translation)
F24:0988             residual = (projection - targets) / 100.0
F24:0989             loss = torch.sqrt(residual.square() + 1e-4).mean()
F24:0990             if not torch.isfinite(loss):
F24:0991                 break
F24:0992             optimizer.zero_grad()
F24:0993             loss.backward()

```

```

F24:0998         optimizer.step()
F24:0999         with torch.no_grad():
F24:1000             parameter[0].clamp_(-3.0, 3.0)
F24:1001             parameter[1].clamp_(-3.0, 3.0)
F24:1002             parameter[2].clamp_(float(np.log(0.1)), float(np.log(5.0)))
F24:1004         with torch.no_grad():
F24:1005             translation = torch.stack(
F24:1006                 (parameter[0], parameter[1], torch.exp(parameter[2]))
F24:1007             )
F24:1008             projection = project(translation)
F24:1009             error = torch.linalg.vector_norm(projection - targets, dim=1).mean()
F24:1010             candidate = (
F24:1011                 float(error),
F24:1012                 translation.detach().clone(),
F24:1013                 projection.detach().clone(),
F24:1014             )
F24:1015             if best is None or candidate[0] < best[0]:
F24:1016                 best = candidate
F24:1018         if best is None:
F24:1019             raise RuntimeError("Camera translation refinement produced no finite solution")
F24:1020         return {
F24:1021             "rotation": identity,
F24:1022             "translation": best[1],
F24:1023             "initial_projection": initial_projection.detach(),
F24:1024             "refined_projection": best[2],
F24:1025             "initial_mean_error_px": float(initial_error),
F24:1026             "refined_mean_error_px": float(best[0]),
F24:1027             "joint_indices": CAMERA_FIT_BODY_INDICES.copy(),
F24:1028         }
F24:1031     def draw_camera_fit_debug(
F24:1032         image_rgb,
F24:1033         target_body_px,
F24:1034         initial_body_px,
F24:1035         refined_body_px,
F24:1036         initial_error,
F24:1037         refined_error,
F24:1038     ):
F24:1039         canvas = image_rgb.copy()
F24:1040         series = (
F24:1041             (target_body_px, (255, 0, 0), "PositionNet"),
F24:1042             (initial_body_px, (0, 0, 255), "network cam"),
F24:1043             (refined_body_px, (0, 255, 0), "refined cam"),
F24:1044         )
F24:1045         for points, color, _ in series:
F24:1046             for start, end in CAMERA_FIT_BODY_EDGES:
F24:1047                 if np.isfinite(points[[start, end]]).all():
F24:1048                     cv2.line(
F24:1049                         canvas,
F24:1050                         tuple(np rint(points[start]).astype(int)),
F24:1051                         tuple(np rint(points[end]).astype(int)),

```



```

F24:1052             color,
F24:1053             4,
F24:1054             cv2.LINE_AA,
F24:1055         )
F24:1056         for index in CAMERA_FIT_BODY_INDICES:
F24:1057             if np.isfinite(points[index]).all():
F24:1058                 cv2.circle(
F24:1059                     canvas,
F24:1060                     tuple(np rint(points[index]).astype(int)),
F24:1061                     8,
F24:1062                     color,
F24:1063                     -1,
F24:1064                     cv2.LINE_AA,
F24:1065                 )
F24:1066         labels = (
F24:1067             ((255, 0, 0), "red: PositionNet target"),
F24:1068             ((0, 0, 255), f"blue: network cam ({initial_error:.1f}px)"),
F24:1069             ((0, 255, 0), f"green: refined cam ({refined_error:.1f}px)"),
F24:1070         )
F24:1071         for row, (color, label) in enumerate(labels):
F24:1072             cv2.putText(
F24:1073                 canvas,
F24:1074                 label,
F24:1075                 (30, 45 + row * 38),
F24:1076                 cv2.FONT_HERSHEY_SIMPLEX,
F24:1077                 0.85,
F24:1078                 color,
F24:1079                 3,
F24:1080                 cv2.LINE_AA,
F24:1081             )
F24:1082         return canvas
F24:1085 def main():
F24:1086     args = parse_args()
F24:1087     paths_to_check = [args.image_dir, args.fisheye_calibration, args.config_path, args.checkpoint_pa
F24:1087+ th]
F24:1088     if not args.no_detector:
F24:1089         paths_to_check.extend([args.det_config, args.det_checkpoint])
F24:1090     for path in paths_to_check:
F24:1091         if not path.exists():
F24:1092             raise FileNotFoundError(path)
F24:1093     if args.no_detector and args.smplx_input_mode != "full_image":
F24:1094         raise ValueError("--no-detector requires --smplx-input-mode full_image")
F24:1095     if (
F24:1096         args.refine_camera_from_positionnet
F24:1097         and args.smplx_input_mode
F24:1098         not in ("full_image", "det_tangent", "det_undistort_contiguous")
F24:1099     ):
F24:1100         raise ValueError(
F24:1101             "Camera refinement requires --smplx-input-mode full_image, "
F24:1102             "det_tangent, or det_undistort_contiguous"

```

```

F24:1103         )
F24:1104     if args.horizontal_crop_pixels < 0:
F24:1105         raise ValueError("--horizontal-crop-pixels must be non-negative")
F24:1106     if args.det_tangent_expand <= 0:
F24:1107         raise ValueError("--det-tangent-expand must be positive")
F24:1108     if args.det_tangent_patch_size <= 0:
F24:1109         raise ValueError("--det-tangent-patch-size must be positive")
F24:1110     if not 0.0 <= args.global_undistort_alpha <= 1.0:
F24:1111         raise ValueError("--global-undistort-alpha must be in [0, 1]")
F24:1112     if (
F24:1113         args.smplx_input_mode in ("det_tangent", "det_undistort_contiguous")
F24:1114         and args.input_crop_mode != "none"
F24:1115     ):
F24:1116         raise ValueError(
F24:1117             f"{args.smplx_input_mode} requires --input-crop-mode none"
F24:1118         )
F24:1121     image_paths = sorted(args.image_dir.glob(args.image_glob))
F24:1122     if args.max_images is not None:
F24:1123         image_paths = image_paths[: args.max_images]
F24:1124     if not image_paths:
F24:1125         raise RuntimeError(f"No images found in {args.image_dir} with glob {args.image_glob}")
F24:1127     output_dir = make_output_dir(args)
F24:1128     overlay_dir = output_dir / "img"
F24:1129     wireframe_dir = output_dir / "wireframe"
F24:1130     smplx_dir = output_dir / "smplx"
F24:1131     meta_dir = output_dir / "meta"
F24:1132     det_dir = output_dir / "detections"
F24:1133     debug_dir = output_dir / "projection_debug"
F24:1134     mesh_dir = output_dir / "mesh"
F24:1135     vit_patch_dir = output_dir / "vit_patches"
F24:1136     positionnet_dir = output_dir / "positionnet"
F24:1137     undistorted_dir = output_dir / "undistorted"
F24:1138     undistorted_detection_dir = output_dir / "undistorted_detections"
F24:1139     joint_compare_dir = output_dir / "joint_compare"
F24:1140     mesh_joint_compare_dir = output_dir / "mesh_joint_compare"
F24:1141     for path in (overlay_dir, smplx_dir, meta_dir, det_dir, debug_dir):
F24:1142         path.mkdir(parents=True, exist_ok=True)
F24:1143     if args.smplx_input_mode in ("det_tangent", "det_undistort_contiguous"):
F24:1144         vit_patch_dir.mkdir(parents=True, exist_ok=True)
F24:1145         positionnet_dir.mkdir(parents=True, exist_ok=True)
F24:1146     if args.smplx_input_mode == "det_undistort_contiguous":
F24:1147         undistorted_dir.mkdir(parents=True, exist_ok=True)
F24:1148         undistorted_detection_dir.mkdir(parents=True, exist_ok=True)
F24:1149     joint_compare_dir.mkdir(parents=True, exist_ok=True)
F24:1150     mesh_joint_compare_dir.mkdir(parents=True, exist_ok=True)
F24:1151     if args.save_mesh:
F24:1152         mesh_dir.mkdir(parents=True, exist_ok=True)
F24:1153     if args.save_wireframe_overlay:
F24:1154         wireframe_dir.mkdir(parents=True, exist_ok=True)
F24:1156     demoer = setup_model(args, output_dir)

```

```

F24:1157     FishEyeCameraCalibrated, smplx_x, load_img, process_bbox, generate_patch_image, render_mesh_fishe
F24:1157+ ye, save_obj = load_runtime_utils()
F24:1158     fisheye_cam = FishEyeCameraCalibrated(str(args.fisheye_calibration.resolve()))
F24:1159     positionnet_capture = {}
F24:1160     if args.smplx_input_mode in ("det_tangent", "det_undistort_contiguous"):
F24:1161         model_core = demoer.model.module if hasattr(demoer.model, "module") else demoer.model
F24:1163         def capture_positionnet_output(_module, _inputs, outputs):
F24:1164             positionnet_capture["body_joint_hm"] = outputs[0].detach().cpu()
F24:1165             positionnet_capture["body_joint_img"] = outputs[1].detach().cpu()
F24:1167             model_core.body_position_net.register_forward_hook(capture_positionnet_output)
F24:1168     detector = None
F24:1169     if not args.no_detector:
F24:1170         detector = init_detector(str(args.det_config.resolve()), str(args.det_checkpoint.resolve()),
F24:1170+ device="cuda:0")
F24:1171     transform = transforms.ToTensor()
F24:1173     first = cv2.imread(str(image_paths[0]), cv2.IMREAD_COLOR)
F24:1174     if first is None:
F24:1175         raise RuntimeError(f"Failed to read {image_paths[0]}")
F24:1176     img_h0, img_w0 = first.shape[:2]
F24:1177     undistort_model = None
F24:1178     if args.smplx_input_mode == "det_undistort_contiguous":
F24:1179         undistort_model = create_global_undistort_model(
F24:1180             args.fisheye_calibration,
F24:1181             img_w0,
F24:1182             img_h0,
F24:1183             args.global_undistort_alpha,
F24:1184         )
F24:1185     video_writer = None
F24:1186     if args.save_video:
F24:1187         fourcc = cv2.VideoWriter_fourcc(*"mp4v")
F24:1188         video_writer = cv2.VideoWriter(str(output_dir / "detected_mesh_overlay_stride10.mp4"), fourc
F24:1188+ c, 10.0, (img_w0, img_h0))
F24:1190     saved_images = []
F24:1191     saved_wireframe_images = []
F24:1192     saved_sampling_grid_images = []
F24:1193     saved_patch_atlas_images = []
F24:1194     saved_positionnet_images = []
F24:1195     saved_camera_fit_images = []
F24:1196     saved_joint_compare_images = []
F24:1197     saved_mesh_joint_compare_images = []
F24:1198     saved_undistorted_images = []
F24:1199     saved_undistorted_detection_images = []
F24:1200     saved_contiguous_patch_grid_images = []
F24:1201     records = []
F24:1202     projection_consistency_errors = []
F24:1203     total_persons = 0
F24:1204     no_detection = 0
F24:1205     start_time = time.time()
F24:1207     progress_desc = "Full-image project" if args.no_detector else "Detect + project"
F24:1208     with tqdm(image_paths, desc=progress_desc, mininterval=10.0) as pbar:

```

```

F24:1209         for image_path in pbar:
F24:1210             original_img = load_img(str(image_path))
F24:1211             vis_img = original_img.copy()
F24:1212             wireframe_img = original_img.copy() if args.save_wireframe_overlay else None
F24:1213             img_h, img_w = original_img.shape[:2]
F24:1214             detector_img = original_img
F24:1215             if args.smplx_input_mode == "det_undistort_contiguous":
F24:1216                 if (img_w, img_h) != (img_w0, img_h0):
F24:1217                     raise ValueError(
F24:1218                         "All globally undistorted frames must share one size: "
F24:1219                         f"expected {(img_w0, img_h0)}, got {(img_w, img_h)}"
F24:1220                     )
F24:1221                 detector_img = undistort_full_image(original_img, undistort_model)
F24:1222                 undistorted_out = undistorted_dir / f"{image_path.stem}_undistorted.jpg"
F24:1223                 cv2.imwrite(str(undistorted_out), rgb_to_bgr_u8(detector_img))
F24:1224                 saved_undistorted_images.append(undistorted_out)
F24:1226             if args.no_detector:
F24:1227                 raw_boxes = []
F24:1228                 det_boxes = [np.array([0.0, 0.0, float(img_w - 1), float(img_h - 1), 1.0], dtype=np.
F24:1228+ float32)]
F24:1229             else:
F24:1230                 if args.smplx_input_mode == "det_undistort_contiguous":
F24:1231                     detector_input = np.ascontiguousarray(rgb_to_bgr_u8(detector_img))
F24:1232                 else:
F24:1233                     detector_input = str(image_path)
F24:1234                 mmdet_results = inference_detector(detector, detector_input)
F24:1235                 raw_boxes = process_mmdet_results(mmdet_results, cat_id=0, multi_person=True)[0]
F24:1236                 filtered = filter_bboxes(raw_boxes, args.score_thr, args.bbox_thr)
F24:1237                 if args.multi_person:
F24:1238                     det_boxes = non_max_suppression(filtered, args.iou_thr) if filtered else []
F24:1239                 else:
F24:1240                     det_boxes = filtered[:1]
F24:1242             if args.smplx_input_mode == "det_undistort_contiguous":
F24:1243                 detector_vis = detector_img.copy()
F24:1244                 for selected_box in det_boxes:
F24:1245                     dx1, dy1, dx2, dy2, dscore = [
F24:1246                         float(value) for value in selected_box[:5]
F24:1247                     ]
F24:1248                     cv2.rectangle(
F24:1249                         detector_vis,
F24:1250                         (int(round(dx1)), int(round(dy1))),
F24:1251                         (int(round(dx2)), int(round(dy2))),
F24:1252                         (255, 0, 0),
F24:1253                         3,
F24:1254                     )
F24:1255                     cv2.putText(
F24:1256                         detector_vis,
F24:1257                         f"person {dscore:.2f}",
F24:1258                         (int(round(dx1)), max(25, int(round(dy1)) - 10)),
F24:1259                         cv2.FONT_HERSHEY_SIMPLEX,

```

```

F24:1260             0.8,
F24:1261             (255, 0, 0),
F24:1262             2,
F24:1263             cv2.LINE_AA,
F24:1264             )
F24:1265             undistorted_detection_out = (
F24:1266                 undistorted_detection_dir
F24:1267                 / f"{image_path.stem}_undistorted_detection.jpg"
F24:1268             )
F24:1269             cv2.imwrite(
F24:1270                 str(undistorted_detection_out),
F24:1271                 rgb_to_bgr_u8(detector_vis),
F24:1272             )
F24:1273             saved_undistorted_detection_images.append(
F24:1274                 undistorted_detection_out
F24:1275             )
F24:1277             det_record = {
F24:1278                 "detector_enabled": not args.no_detector,
F24:1279                 "image_path": str(image_path.resolve()),
F24:1280                 "raw_person_detections": int(len(raw_boxes)),
F24:1281                 "score_thr": float(args.score_thr),
F24:1282                 "bbox_thr": float(args.bbox_thr),
F24:1283                 "detector_coordinate_space": (
F24:1284                     "global_undistorted"
F24:1285                     if args.smplx_input_mode == "det_undistort_contiguous"
F24:1286                     else "original_fisheye"
F24:1287                 ),
F24:1288                 "selected_bboxes_xyxy_score": [np.asarray(b).astype(float).tolist() for b in det_box
F24:1289+ es],
F24:1289             }
F24:1290             (det_dir / f"{image_path.stem}_detections.json").write_text(json.dumps(det_record, inden
F24:1290+ t=2) + "\n")
F24:1292             if not det_boxes:
F24:1293                 no_detection += 1
F24:1294                 vis_img = draw_no_detection(vis_img)
F24:1295                 out_img = overlay_dir / f"{image_path.stem}_det_mesh_overlay.jpg"
F24:1296                 cv2.imwrite(str(out_img), rgb_to_bgr_u8(vis_img))
F24:1297                 saved_images.append(out_img)
F24:1298                 if wireframe_img is not None:
F24:1299                     wireframe_out = wireframe_dir / f"{image_path.stem}_det_mesh_wireframe.jpg"
F24:1300                     cv2.imwrite(str(wireframe_out), rgb_to_bgr_u8(wireframe_img))
F24:1301                     saved_wireframe_images.append(wireframe_out)
F24:1302                 if video_writer is not None:
F24:1303                     video_writer.write(rgb_to_bgr_u8(vis_img))
F24:1304                 records.append(**det_record, "persons_projected": 0)
F24:1305                 continue
F24:1307             persons_projected = 0
F24:1308             for bbox_id, det_box in enumerate(det_boxes):
F24:1309                 x1, y1, x2, y2, score = [float(v) for v in det_box[:5]]
F24:1310                 mmdet_box_xywh = np.array([x1, y1, abs(x2 - x1), abs(y2 - y1)], dtype=np.float32)

```

```

F24:1311         sampling_bbox_xyxy = None
F24:1312         sampling_grid_pixels = None
F24:1313         if args.smplx_input_mode == "det_tangent":
F24:1314             sampling_bbox_xyxy = expand_bbox_xyxy(
F24:1315                 det_box,
F24:1316                 args.det_tangent_expand,
F24:1317                 img_w,
F24:1318                 img_h,
F24:1319             )
F24:1320             sx1, syl, sx2, sy2 = sampling_bbox_xyxy
F24:1321             bbox = np.asarray([sx1, syl, sx2 - sx1, sy2 - syl], dtype=np.float32)
F24:1322         elif args.smplx_input_mode == "det_undistort_contiguous":
F24:1323             if args.contiguous_roi_mode == "stretch":
F24:1324                 sampling_bbox_xyxy = expand_bbox_xyxy(
F24:1325                     det_box,
F24:1326                     args.det_tangent_expand,
F24:1327                     img_w,
F24:1328                     img_h,
F24:1329                 )
F24:1330             else:
F24:1331                 sampling_bbox_xyxy = expand_bbox_to_aspect(
F24:1332                     det_box,
F24:1333                     args.det_tangent_expand,
F24:1334                     target_width_over_height=12.0 / 16.0,
F24:1335                 )
F24:1336                 sx1, syl, sx2, sy2 = sampling_bbox_xyxy
F24:1337                 bbox = np.asarray(
F24:1338                     [sx1, syl, sx2 - sx1, sy2 - syl],
F24:1339                     dtype=np.float32,
F24:1340                 )
F24:1341             else:
F24:1342                 bbox = process_bbox(mmdet_box_xywh, img_w, img_h)
F24:1343             if bbox is None:
F24:1344                 continue
F24:1345             if args.smplx_input_mode == "det_crop":
F24:1346                 img_crop, img2bb_trans, bb2img_trans = generate_patch_image(
F24:1347                     original_img, bbox, 1.0, 0.0, False, cfg.input_img_shape
F24:1348                 )
F24:1349             img = transform(img_crop.astype(np.float32)) / 255.0
F24:1350             img = img.cuda() [None, :, :, :]
F24:1351             meta_info = {}
F24:1352             # FishEyeCameraCalibrated.world2camera already returns
F24:1353             # original fisheye-image pixel coordinates. Applying
F24:1354             # bb2img_trans here double-applies the crop inverse and
F24:1355             # shifts/scales the mesh overlay.
F24:1356             render_proj_trans = None
F24:1357         else:
F24:1358             img2bb_trans = np.eye(2, 3, dtype=np.float32)
F24:1359             bb2img_trans = np.eye(2, 3, dtype=np.float32)
F24:1360             input_img = (
F24:1361

```

```

F24:1362         detector_img
F24:1363         if args.smplx_input_mode == "det_undistort_contiguous"
F24:1364         else original_img
F24:1365     )
F24:1366     input_offset = np.array([0.0, 0.0], dtype=np.float32)
F24:1367     input_crop_xyxy = [0, 0, int(img_w), int(img_h)]
F24:1368     if args.input_crop_mode == "center_square":
F24:1369         side = min(img_h, img_w)
F24:1370         crop_left = int(round((img_w - side) / 2.0))
F24:1371         crop_top = int(round((img_h - side) / 2.0))
F24:1372         input_img = original_img[crop_top:crop_top + side, crop_left:crop_left + sid
F24:1372+ e, :]
F24:1373         input_offset = np.array([crop_left, crop_top], dtype=np.float32)
F24:1374         input_crop_xyxy = [crop_left, crop_top, crop_left + side, crop_top + side]
F24:1375     elif args.input_crop_mode == "training_horizontal":
F24:1376         crop_left = int(args.horizontal_crop_pixels)
F24:1377         crop_right = int(args.horizontal_crop_pixels)
F24:1378         if crop_left + crop_right >= img_w:
F24:1379             raise ValueError(
F24:1380                 "Horizontal crop removes the entire image: "
F24:1381                 f"width={img_w}, left={crop_left}, right={crop_right}"
F24:1382             )
F24:1383         input_img = original_img[:, crop_left:img_w - crop_right, :]
F24:1384         input_offset = np.array([crop_left, 0.0], dtype=np.float32)
F24:1385         input_crop_xyxy = [crop_left, 0, img_w - crop_right, img_h]
F24:1386     input_h, input_w = input_img.shape[:2]
F24:1387     img = transform(input_img.astype(np.float32)) / 255.0
F24:1388     img = img.cuda() [None, :, :, :]
F24:1389     meta_info = {
F24:1390         "camera_id": torch.zeros((1,), dtype=torch.long, device=img.device),
F24:1391         "input_img_shape": torch.tensor([[input_h, input_w]], dtype=torch.float32, d
F24:1391+ evice=img.device),
F24:1392         "input_coord_offset": torch.from_numpy(input_offset[None]).to(device=img.dev
F24:1392+ ice, dtype=torch.float32),
F24:1393     }
F24:1394     render_proj_trans = None
F24:1395     if args.smplx_input_mode in (
F24:1396         "det_tangent",
F24:1397         "det_undistort_contiguous",
F24:1398     ):
F24:1399         model_core = demoer.model.module if hasattr(demoer.model, "module") else dem
F24:1399+ oer.model
F24:1400         sampler = model_core.encoder.fisheye2sphere
F24:1401         patch_rows = int(sampler.patch_num_vertical)
F24:1402         patch_columns = int(sampler.selected_patch_indices.numel() // patch_rows)
F24:1403         if patch_rows * patch_columns != 192:
F24:1404             raise ValueError(
F24:1405                 "Detection-aligned sampling expects the checkpoint's "
F24:1406                 "192-token 16-row x 12-column ViT layout; "
F24:1407                 f"got rows={patch_rows}, columns={patch_columns}"

```

```
F24:1408         )
F24:1409     if args.smplx_input_mode == "det_tangent":
F24:1410         sampling_grid_pixels = create_bbox_tangent_pixel_grid(
F24:1411             sampler,
F24:1412             fisheye_cam,
F24:1413             sampling_bbox_xyxy,
F24:1414             patch_rows=patch_rows,
F24:1415             patch_columns=patch_columns,
F24:1416             patch_size=args.det_tangent_patch_size,
F24:1417         )
F24:1418     else:
F24:1419         patch_h, patch_w = [
F24:1420             int(value)
F24:1421             for value in sampler.patches_2d_per_camera_pixels.shape[-3:-1]
F24:1422         ]
F24:1423         sampling_grid_pixels = create_contiguous_bbox_pixel_grid(
F24:1424             sampling_bbox_xyxy,
F24:1425             patch_rows=patch_rows,
F24:1426             patch_columns=patch_columns,
F24:1427             patch_h=patch_h,
F24:1428             patch_w=patch_w,
F24:1429         )
F24:1430         sampler.patches_2d_per_camera_pixels = torch.from_numpy(
F24:1431             sampling_grid_pixels[None]
F24:1432         ).to(device=img.device, dtype=img.dtype)
F24:1433         sampler.selected_patch_indices = torch.arange(
F24:1434             sampling_grid_pixels.shape[0],
F24:1435             device=img.device,
F24:1436             dtype=torch.long,
F24:1437         )
F24:1438         with torch.no_grad():
F24:1439             sampled_vit_patches = sampler(
F24:1440                 img,
F24:1441                 camera_ids=meta_info["camera_id"],
F24:1442                 input_img_shapes=meta_info["input_img_shape"],
F24:1443                 input_coord_offsets=meta_info["input_coord_offset"],
F24:1444             )
F24:1445         patch_prefix = f"{image_path.stem}_{bbox_id:02d}"
F24:1446         np.save(
F24:1447             vit_patch_dir / f"{patch_prefix}_sampling_grid_pixels.npy",
F24:1448             sampling_grid_pixels,
F24:1449         )
F24:1450         if args.smplx_input_mode == "det_tangent":
F24:1451             sampling_overlay = draw_tangent_sampling_grid(
F24:1452                 original_img,
F24:1453                 sampling_grid_pixels,
F24:1454                 sampling_bbox_xyxy,
F24:1455             )
F24:1456         else:
F24:1457             sampling_overlay = draw_contiguous_sampling_grid(
```



```

F24:1458             detector_img,
F24:1459             sampling_grid_pixels,
F24:1460             sampling_bbox_xyxy,
F24:1461         )
F24:1462         sampling_grid_out = vit_patch_dir / f"{patch_prefix}_sampling_grid.jpg"
F24:1463         patch_atlas_out = vit_patch_dir / f"{patch_prefix}_patch_atlas.jpg"
F24:1464         cv2.imwrite(str(sampling_grid_out), rgb_to_bgr_u8(sampling_overlay))
F24:1465         if args.smplx_input_mode == "det_tangent":
F24:1466             save_vit_patch_atlas(
F24:1467                 sampled_vit_patches,
F24:1468                 patch_atlas_out,
F24:1469                 rows=patch_rows,
F24:1470                 columns=patch_columns,
F24:1471             )
F24:1472         else:
F24:1473             save_vit_patch_atlas(
F24:1474                 sampled_vit_patches,
F24:1475                 patch_atlas_out,
F24:1476                 rows=patch_rows,
F24:1477                 columns=patch_columns,
F24:1478                 gap=0,
F24:1479             )
F24:1480         patch_grid_out = (
F24:1481             vit_patch_dir
F24:1482             / f"{patch_prefix}_patch_atlas_grid.jpg"
F24:1483         )
F24:1484         save_vit_patch_atlas(
F24:1485             sampled_vit_patches,
F24:1486             patch_grid_out,
F24:1487             rows=patch_rows,
F24:1488             columns=patch_columns,
F24:1489             gap=0,
F24:1490             grid_color_rgb=(255, 0, 0),
F24:1491         )
F24:1492         saved_contiguous_patch_grid_images.append(
F24:1493             patch_grid_out
F24:1494         )
F24:1495         saved_sampling_grid_images.append(sampling_grid_out)
F24:1496         saved_patch_atlas_images.append(patch_atlas_out)
F24:1497         positionnet_capture.clear()
F24:1498         with torch.no_grad():
F24:1499             out = demoer.model({"img": img, "img_ori": img}, {}, meta_info, "test")
F24:1500         body_joint_pixels_undistorted = None
F24:1501         if args.smplx_input_mode in (
F24:1502             "det_tangent",
F24:1503             "det_undistort_contiguous",
F24:1504         ):
F24:1505             if not {"body_joint_hm", "body_joint_img"}.issubset(positionnet_capture):
F24:1506                 raise RuntimeError("PositionNet forward hook did not capture its outputs")
F24:1507             body_joint_hm = positionnet_capture["body_joint_hm"].numpy()[0]

```

```

F24:1508         body_joint_img = positionnet_capture["body_joint_img"].numpy()[0]
F24:1509         body_joint_pixels_sampling, patch_center_grid_pixels = (
F24:1510             positionnet_coords_to_sampling_grid(
F24:1511                 body_joint_img,
F24:1512                 sampling_grid_pixels,
F24:1513                 patch_rows=patch_rows,
F24:1514                 patch_columns=patch_columns,
F24:1515             )
F24:1516         )
F24:1517         if args.smplx_input_mode == "det_undistort_contiguous":
F24:1518             body_joint_pixels_undistorted = body_joint_pixels_sampling
F24:1519             body_joint_pixels = undistorted_pixels_to_original(
F24:1520                 body_joint_pixels_undistorted,
F24:1521                 undistort_model,
F24:1522             )
F24:1523             positionnet_canvas = detector_img
F24:1524             positionnet_canvas_points = body_joint_pixels_undistorted
F24:1525         else:
F24:1526             body_joint_pixels = body_joint_pixels_sampling
F24:1527             positionnet_canvas = original_img
F24:1528             positionnet_canvas_points = body_joint_pixels
F24:1529         positionnet_prefix = f"{image_path.stem}_{bbox_id:02d}"
F24:1530         positionnet_payload = {
F24:1531             "body_joint_hm": body_joint_hm.astype(np.float32),
F24:1532             "body_joint_img": body_joint_img.astype(np.float32),
F24:1533             "body_joint_xy_original_from_grid": body_joint_pixels.astype(np.float32),
F24:1534             "patch_center_grid_pixels": patch_center_grid_pixels.astype(np.float32),
F24:1535             "body_joint_names": np.asarray(smpl_x.pos_joints_name[:25]),
F24:1536             "sampling_bbox_xyxy": sampling_bbox_xyxy.astype(np.float32),
F24:1537             "output_hm_shape": np.asarray(cfg.output_hm_shape, dtype=np.int32),
F24:1538         }
F24:1539         if body_joint_pixels_undistorted is not None:
F24:1540             positionnet_payload[
F24:1541                 "body_joint_xy_global_undistorted"
F24:1542             ] = body_joint_pixels_undistorted.astype(np.float32)
F24:1543             positionnet_payload[
F24:1544                 "global_undistorted_new_k"
F24:1545             ] = undistort_model["new_k"].astype(np.float64)
F24:1546         np.savez_compressed(
F24:1547             positionnet_dir / f"{positionnet_prefix}_positionnet.npz",
F24:1548             **positionnet_payload,
F24:1549         )
F24:1550         positionnet_vis = draw_positionnet_result(
F24:1551             positionnet_canvas,
F24:1552             body_joint_hm,
F24:1553             positionnet_canvas_points,
F24:1554             sampling_bbox_xyxy,
F24:1555         )
F24:1556         positionnet_vis_out = positionnet_dir / f"{positionnet_prefix}_positionnet.jpg"
F24:1557         cv2.imwrite(str(positionnet_vis_out), rgb_to_bgr_u8(positionnet_vis))

```

```

F24:1558         saved_positionnet_images.append(positionnet_vis_out)
F24:1559         mesh = out["smplx_mesh_cam"].detach().cpu().numpy()[0]
F24:1560         network_translation = out["cam_trans"].detach()[0]
F24:1561         render_translation = network_translation
F24:1562         network_global_orient = out["smplx_root_pose"].detach()[0]
F24:1563         render_global_orient = network_global_orient.clone()
F24:1564         identity_rotation = torch.eye(
F24:1565             3,
F24:1566             device=network_global_orient.device,
F24:1567             dtype=network_global_orient.dtype,
F24:1568         )
F24:1569         global_orient_delta_matrix = identity_rotation.clone()
F24:1570         camera_rotation_delta = identity_rotation.clone()
F24:1571         global_orient_delta_degrees = 0.0
F24:1572         global_orient_mesh_consistency_max_error_m = None
F24:1573         global_orient_stage_mean_error_px = None
F24:1574         body_targets_px = None
F24:1575         body_projection_initial_px = None
F24:1576         body_projection_refined_px = None
F24:1577         body_projection_model_px = None
F24:1578         projection_consistency_max_error_px = None
F24:1579         camera_fit = None
F24:1581         if args.smplx_input_mode in (
F24:1582             "full_image",
F24:1583             "det_tangent",
F24:1584             "det_undistort_contiguous",
F24:1585         ):
F24:1586             # smplx_joint_cam_pred is root-relative and is intended for
F24:1587             # 3D evaluation. Rebuild the absolute local joints from the
F24:1588             # predicted SMPL-X parameters before adding cam_trans; these
F24:1589             # are the joints used by get_coord to create joint_proj.
F24:1590             zero_eye_pose = torch.zeros_like(out["smplx_jaw_pose"])
F24:1591             with torch.no_grad():
F24:1592                 model_core = demoer.model.module if hasattr(demoer.model, "module") else dem
F24:1592+ oer.model
F24:1593                 reconstructed_smplx = model_core.smplx_layer(
F24:1594                     betas=out["smplx_shape"],
F24:1595                     body_pose=out["smplx_body_pose"],
F24:1596                     global_orient=out["smplx_root_pose"],
F24:1597                     right_hand_pose=out["smplx_rhand_pose"],
F24:1598                     left_hand_pose=out["smplx_lhand_pose"],
F24:1599                     jaw_pose=out["smplx_jaw_pose"],
F24:1600                     leye_pose=zero_eye_pose,
F24:1601                     reye_pose=zero_eye_pose,
F24:1602                     expression=out["smplx_expr"],
F24:1603                 )
F24:1604                 body_joints_local = reconstructed_smplx.joints[0, smplx.joint_idx[:25], :]
F24:1605                 body_joints_local_render = body_joints_local
F24:1606                 if args.smplx_input_mode in (
F24:1607                     "det_tangent",

```

```

F24:1608         "det_undistort_contiguous",
F24:1609     ):
F24:1610         body_targets_px = body_joint_pixels
F24:1611     else:
F24:1612         joint_img = out["joint_img"].detach().cpu().numpy()[0]
F24:1613         body_targets_px = heatmap_joints_to_original_pixels(
F24:1614             joint_img[:25, :2], (input_h, input_w), input_offset
F24:1615         )
F24:1616     with torch.no_grad():
F24:1617         body_projection_initial = fisheye_cam.world2camera_pytorch(
F24:1618             body_joints_local + network_translation[None, :]
F24:1619         )
F24:1620         body_projection_initial_px = body_projection_initial.cpu().numpy()
F24:1621         body_projection_refined_px = body_projection_initial_px.copy()
F24:1622         body_projection_model_px = heatmap_joints_to_original_pixels(
F24:1623             out["smplx_joint_proj"].detach().cpu().numpy()[0, :25, :2],
F24:1624             (input_h, input_w),
F24:1625             input_offset,
F24:1626         )
F24:1627         projection_consistency_max_error_px = float(
F24:1628             np.linalg.norm(
F24:1629                 body_projection_model_px - body_projection_initial_px,
F24:1630                 axis=1,
F24:1631             ).max()
F24:1632         )
F24:1633         projection_consistency_errors.append(projection_consistency_max_error_px)
F24:1635     if args.refine_camera_from_positionnet:
F24:1636         camera_fit_fn = (
F24:1637             refine_camera_translation
F24:1638             if args.camera_fit_mode == "translation"
F24:1639             else refine_camera_extrinsics
F24:1640         )
F24:1641         camera_fit = camera_fit_fn(
F24:1642             body_joints_local,
F24:1643             body_targets_px,
F24:1644             network_translation,
F24:1645             fisheye_cam,
F24:1646             args.camera_fit_iters,
F24:1647             args.camera_fit_lr,
F24:1648         )
F24:1649         render_rotation = camera_fit["rotation"]
F24:1650         render_translation = camera_fit["translation"]
F24:1651         if args.camera_fit_mode == "translation_global_orient":
F24:1652             global_orient_stage_mean_error_px = camera_fit[
F24:1653                 "refined_mean_error_px"
F24:1654             ]
F24:1655             global_orient_delta_matrix = render_rotation
F24:1656             render_global_orient = compose_global_orient(
F24:1657                 render_rotation,
F24:1658                 network_global_orient,

```

```

F24:1659         )
F24:1660         cosine = ((torch.trace(render_rotation) - 1.0) * 0.5).clamp(-1.0, 1.0)
F24:1661         global_orient_delta_degrees = float(
F24:1662             torch.rad2deg(torch.acos(cosine)).item()
F24:1663         )
F24:1664         with torch.no_grad():
F24:1665             refined_smplx = model_core.smplx_layer(
F24:1666                 betas=out["smplx_shape"],
F24:1667                 body_pose=out["smplx_body_pose"],
F24:1668                 global_orient=render_global_orient[None, :],
F24:1669                 right_hand_pose=out["smplx_rhand_pose"],
F24:1670                 left_hand_pose=out["smplx_lhand_pose"],
F24:1671                 jaw_pose=out["smplx_jaw_pose"],
F24:1672                 leye_pose=zero_eye_pose,
F24:1673                 reye_pose=zero_eye_pose,
F24:1674                 expression=out["smplx_expr"],
F24:1675             )
F24:1676         body_joints_local_render = refined_smplx.joints[
F24:1677             0, smplx.joint_idx[:25], :
F24:1678         ]
F24:1679         translation_refit = refine_camera_translation(
F24:1680             body_joints_local_render,
F24:1681             body_targets_px,
F24:1682             render_translation,
F24:1683             fisheye_cam,
F24:1684             args.camera_fit_iters,
F24:1685             args.camera_fit_lr,
F24:1686         )
F24:1687         render_translation = translation_refit["translation"]
F24:1688         camera_fit["translation"] = render_translation
F24:1689         camera_fit["refined_projection"] = translation_refit[
F24:1690             "refined_projection"
F24:1691         ]
F24:1692         camera_fit["refined_mean_error_px"] = translation_refit[
F24:1693             "refined_mean_error_px"
F24:1694         ]
F24:1695         mesh = (
F24:1696             refined_smplx.vertices[0]
F24:1697             + render_translation[None, :]
F24:1698         ).detach().cpu().numpy()
F24:1699         mesh_from_refined_params = mesh.copy()
F24:1700         global_orient_mesh_consistency_max_error_m = float(
F24:1701             np.linalg.norm(
F24:1702                 mesh - mesh_from_refined_params,
F24:1703                 axis=1,
F24:1704             ).max()
F24:1705         )
F24:1706         elif args.camera_fit_mode == "extrinsics":
F24:1707             camera_rotation_delta = render_rotation
F24:1708             body_joints_local_render = (

```

```

F24:1709             body_joints_local @ render_rotation.t()
F24:1710         )
F24:1711     else:
F24:1712         body_joints_local_render = body_joints_local
F24:1713     with torch.no_grad():
F24:1714         body_projection_refined = fisheye_cam.world2camera_pytorch(
F24:1715             body_joints_local_render
F24:1716             + render_translation[None, :])
F24:1717     )
F24:1718     body_projection_refined_px = body_projection_refined.cpu().numpy()
F24:1719     if args.camera_fit_mode != "translation_global_orient":
F24:1720         network_translation_np = network_translation.cpu().numpy()
F24:1721         render_translation_np = render_translation.cpu().numpy()
F24:1722         mesh = (
F24:1723             (mesh - network_translation_np[None, :])
F24:1724             @ render_rotation.cpu().numpy().T
F24:1725             + render_translation_np[None, :])
F24:1726     )
F24:1727     debug_image = draw_camera_fit_debug(
F24:1728         original_img,
F24:1729         body_targets_px,
F24:1730         body_projection_initial_px,
F24:1731         body_projection_refined_px,
F24:1732         camera_fit["initial_mean_error_px"],
F24:1733         camera_fit["refined_mean_error_px"],
F24:1734     )
F24:1735     camera_fit_out = debug_dir / f"{image_path.stem}_{bbox_id:02d}_camera_fit.jp
F24:1735+ g"
F24:1736     cv2.imwrite(str(camera_fit_out), rgb_to_bgr_u8(debug_image))
F24:1737     saved_camera_fit_images.append(camera_fit_out)
F24:1739     if body_targets_px is not None:
F24:1740         joint_compare_image = draw_positionnet_smplx_2d_comparison(
F24:1741             original_img,
F24:1742             body_targets_px,
F24:1743             body_projection_refined_px,
F24:1744         )
F24:1745         joint_compare_out = (
F24:1746             joint_compare_dir
F24:1747             / f"{image_path.stem}_{bbox_id:02d}_positionnet_vs_smplx.jpg"
F24:1748         )
F24:1749         cv2.imwrite(
F24:1750             str(joint_compare_out),
F24:1751             rgb_to_bgr_u8(joint_compare_image),
F24:1752         )
F24:1753         saved_joint_compare_images.append(joint_compare_out)
F24:1755     smplx_pred = {
F24:1756         "global_orient": render_global_orient.reshape(-1, 3).detach().cpu().numpy(),
F24:1757         "global_orient_network": network_global_orient.reshape(-1, 3).detach().num
F24:1757+ py(),
F24:1758         "global_orient_delta_matrix": global_orient_delta_matrix.reshape(1, 3, 3).detach

```

```

F24:1758+ ().cpu().numpy(),
F24:1759         "body_pose": out["smplx_body_pose"].reshape(-1, 3).detach().cpu().numpy(),
F24:1760         "left_hand_pose": out["smplx_lhand_pose"].reshape(-1, 3).detach().cpu().numpy(),
F24:1761         "right_hand_pose": out["smplx_rhand_pose"].reshape(-1, 3).detach().cpu().numpy()
F24:1761+ ,
F24:1762         "jaw_pose": out["smplx_jaw_pose"].reshape(-1, 3).detach().cpu().numpy(),
F24:1763         "leye_pose": np.zeros((1, 3), dtype=np.float32),
F24:1764         "reye_pose": np.zeros((1, 3), dtype=np.float32),
F24:1765         "betas": out["smplx_shape"].reshape(-1, 10).detach().cpu().numpy(),
F24:1766         "expression": out["smplx_expr"].reshape(-1, 10).detach().cpu().numpy(),
F24:1767         "transl": render_translation.reshape(-1, 3).detach().cpu().numpy(),
F24:1768         "transl_network": network_translation.reshape(-1, 3).detach().cpu().numpy(),
F24:1769         "camera_rotation_delta": camera_rotation_delta.reshape(1, 3, 3).detach().cpu().n
F24:1769+ umpy(),
F24:1770         "det_bbox_xyxy_score": np.asarray(det_box).reshape(1, 5).astype(np.float32),
F24:1771         "smplx_joint_cam_root_relative": out["smplx_joint_cam_pred"][0].detach().cpu().n
F24:1771+ umpy().astype(np.float32),
F24:1772         "smplx_mesh_cam": mesh.astype(np.float32),
F24:1773     }
F24:1774     if body_targets_px is not None:
F24:1775         body_joints_camera_network = body_joints_local + network_translation[None, :]
F24:1776         if camera_fit is not None:
F24:1777             body_joints_camera_render = (
F24:1778                 body_joints_local_render + render_translation[None, :]
F24:1779             )
F24:1780         else:
F24:1781             body_joints_camera_render = body_joints_camera_network
F24:1782         smplx_pred["smplx_body_joints_3d_local"] = body_joints_local.detach().cpu().num
F24:1782+ y().astype(np.float32)
F24:1783         if args.camera_fit_mode == "translation_global_orient" and camera_fit is not Non
F24:1783+ e:
F24:1784             smplx_pred["smplx_body_joints_3d_local_refined_global_orient"] = (
F24:1785                 body_joints_local_render
F24:1786             ).detach().cpu().numpy().astype(np.float32)
F24:1787         smplx_pred["smplx_body_joints_3d_network_cam"] = body_joints_camera_network.deta
F24:1787+ ch().cpu().numpy().astype(np.float32)
F24:1788         smplx_pred["smplx_body_joints_3d_render_cam"] = body_joints_camera_render.detach
F24:1788+ ().cpu().numpy().astype(np.float32)
F24:1789         smplx_pred["positionnet_body_joints_2d"] = body_targets_px.astype(np.float32)
F24:1790         if body_joint_pixels_undistorted is not None:
F24:1791             smplx_pred[
F24:1792                 "positionnet_body_joints_2d_global_undistorted"
F24:1793             ] = body_joint_pixels_undistorted.astype(np.float32)
F24:1794         smplx_pred["smplx_body_joints_2d_network_cam"] = body_projection_initial_px.astyp
F24:1794+ pe(np.float32)
F24:1795         smplx_pred["smplx_body_joints_2d_render_cam"] = body_projection_refined_px.astyp
F24:1795+ e(np.float32)
F24:1796         smplx_pred["smplx_body_joints_2d_model_output"] = body_projection_model_px.astyp
F24:1796+ e(np.float32)
F24:1797         smplx_pred["projection_consistency_max_error_px"] = np.asarray(

```

```

F24:1798         projection_consistency_max_error_px, dtype=np.float32
F24:1799     )
F24:1800     np.savez(smplx_dir / f"{image_path.stem}_{bbox_id:02d}_smplx.npz", **smplx_pred)
F24:1801     if args.save_mesh:
F24:1802         save_obj(mesh, smpl_x.face, str(mesh_dir / f"{image_path.stem}_{bbox_id:02d}.obj
F24:1802+ "))
F24:1804     vis_img = render_mesh_fisheye(
F24:1805         vis_img,
F24:1806         mesh,
F24:1807         smpl_x.face,
F24:1808         {"focal": [1.0, 1.0], "princpt": [0.0, 0.0]},
F24:1809         fisheye_cam,
F24:1810         mesh_as_vertices=args.show_verts,
F24:1811         mode=args.mesh_mode,
F24:1812         proj_trans=render_proj_trans,
F24:1813     )
F24:1814     if wireframe_img is not None:
F24:1815         wireframe_img = render_mesh_fisheye(
F24:1816             wireframe_img,
F24:1817             mesh,
F24:1818             smpl_x.face,
F24:1819             {"focal": [1.0, 1.0], "princpt": [0.0, 0.0]},
F24:1820             fisheye_cam,
F24:1821             mesh_as_vertices=False,
F24:1822             mode="wireframe",
F24:1823             proj_trans=render_proj_trans,
F24:1824         )
F24:1825     if body_targets_px is not None:
F24:1826         mesh_joint_compare_image = draw_positionnet_smplx_2d_comparison(
F24:1827             wireframe_img,
F24:1828             body_targets_px,
F24:1829             body_projection_refined_px,
F24:1830         )
F24:1831         mesh_joint_compare_out = (
F24:1832             mesh_joint_compare_dir
F24:1833             / f"{image_path.stem}_{bbox_id:02d}_mesh_positionnet_smplx.jpg"
F24:1834         )
F24:1835         cv2.imwrite(
F24:1836             str(mesh_joint_compare_out),
F24:1837             rgb_to_bgr_u8(mesh_joint_compare_image),
F24:1838         )
F24:1839         saved_mesh_joint_compare_images.append(mesh_joint_compare_out)
F24:1840     if args.show_bbox and not args.no_detector:
F24:1841         if args.smplx_input_mode == "det_undistort_contiguous":
F24:1842             det_corners_undistorted = np.asarray(
F24:1843                 [[x1, y1], [x2, y1], [x2, y2], [x1, y2]],
F24:1844                 dtype=np.float32,
F24:1845             )
F24:1846             det_corners_original = undistorted_pixels_to_original(
F24:1847                 det_corners_undistorted,

```



```

F24:1848             undistort_model,
F24:1849         )
F24:1850         det_corners_int = np rint(
F24:1851             det_corners_original
F24:1852         ).astype(np.int32)
F24:1853         cv2.polylines(
F24:1854             vis_img,
F24:1855             [det_corners_int],
F24:1856             isClosed=True,
F24:1857             color=(255, 0, 0),
F24:1858             thickness=3,
F24:1859             lineType=cv2.LINE_AA,
F24:1860         )
F24:1861         label_point = tuple(det_corners_int[0])
F24:1862     else:
F24:1863         pt1 = (int(round(x1)), int(round(y1)))
F24:1864         pt2 = (int(round(x2)), int(round(y2)))
F24:1865         cv2.rectangle(vis_img, pt1, pt2, (255, 0, 0), 3)
F24:1866         label_point = pt1
F24:1867     cv2.putText(
F24:1868         vis_img,
F24:1869         f"person {score:.2f}",
F24:1870         (label_point[0], max(25, label_point[1] - 10)),
F24:1871         cv2.FONT_HERSHEY_SIMPLEX,
F24:1872         0.8,
F24:1873         (255, 0, 0),
F24:1874         2,
F24:1875         cv2.LINE_AA,
F24:1876     )
F24:1877     meta = {
F24:1878         "image_path": str(image_path.resolve()),
F24:1879         "bbox_id": int(bbox_id),
F24:1880         "det_bbox_xyxy_score": np.asarray(det_box).astype(float).tolist(),
F24:1881         "bbox_processed_xywh": bbox.tolist(),
F24:1882         "img2bb_trans": img2bb_trans.tolist(),
F24:1883         "bb2img_trans": bb2img_trans.tolist(),
F24:1884         "fisheye_calibration": str(args.fisheye_calibration.resolve()),
F24:1885         "checkpoint_path": str(args.checkpoint_path.resolve()),
F24:1886         "config_path": str(args.config_path.resolve()),
F24:1887         "smplx_input_mode": args.smplx_input_mode,
F24:1888         "detector_coordinate_space": (
F24:1889             "global_undistorted"
F24:1890             if args.smplx_input_mode == "det_undistort_contiguous"
F24:1891             else "original_fisheye"
F24:1892         ),
F24:1893         "input_crop_mode": args.input_crop_mode,
F24:1894         "input_crop_xyxy": input_crop_xyxy if args.smplx_input_mode != "det_crop" else N
F24:1895+ one,
F24:1896         "input_coord_offset": input_offset.astype(float).tolist() if args.smplx_input_mo
F24:1896+ de != "det_crop" else [0.0, 0.0],

```

```

F24:1897         "det_tangent_sampling_bbox_xyxy": (
F24:1898             sampling_bbox_xyxy.astype(float).tolist()
F24:1899             if sampling_bbox_xyxy is not None
F24:1900             else None
F24:1901         ),
F24:1902         "sampling_bbox_coordinate_space": (
F24:1903             "global_undistorted"
F24:1904             if args.smplx_input_mode == "det_undistort_contiguous"
F24:1905             else (
F24:1906                 "original_fisheye"
F24:1907                 if sampling_bbox_xyxy is not None
F24:1908                 else None
F24:1909             )
F24:1910         ),
F24:1911         "det_tangent_patch_grid_shape": (
F24:1912             list(sampling_grid_pixels.shape)
F24:1913             if sampling_grid_pixels is not None
F24:1914             else None
F24:1915         ),
F24:1916         "det_tangent_patch_size": (
F24:1917             float(args.det_tangent_patch_size)
F24:1918             if args.smplx_input_mode == "det_tangent"
F24:1919             else None
F24:1920         ),
F24:1921         "positionnet_2d_mapping": (
F24:1922             "bilinear_tangent_patch_centers"
F24:1923             if args.smplx_input_mode == "det_tangent"
F24:1924             else (
F24:1925                 "continuous_undistorted_grid_then_brown_conrady_to_original"
F24:1926                 if args.smplx_input_mode == "det_undistort_contiguous"
F24:1927                 else "heatmap_to_full_image"
F24:1928             )
F24:1929         ),
F24:1930         "global_undistort_alpha": (
F24:1931             float(args.global_undistort_alpha)
F24:1932             if args.smplx_input_mode == "det_undistort_contiguous"
F24:1933             else None
F24:1934         ),
F24:1935         "contiguous_roi_mode": (
F24:1936             args.contiguous_roi_mode
F24:1937             if args.smplx_input_mode == "det_undistort_contiguous"
F24:1938             else None
F24:1939         ),
F24:1940         "global_undistorted_new_k": (
F24:1941             undistort_model["new_k"].astype(float).tolist()
F24:1942             if args.smplx_input_mode == "det_undistort_contiguous"
F24:1943             else None
F24:1944         ),
F24:1945         "camera_translation_network": network_translation.detach().cpu().numpy().astype(
F24:1945+ float).tolist(),

```

```

F24:1946             "camera_translation_render": render_translation.detach().cpu().numpy().astype(float)
F24:1946+ oat).tolist(),
F24:1947             "global_orient_network": network_global_orient.detach().cpu().numpy().astype(float)
F24:1947+ at).tolist(),
F24:1948             "global_orient_render": render_global_orient.detach().cpu().numpy().astype(float)
F24:1948+ ).tolist(),
F24:1949             "global_orient_delta_matrix": global_orient_delta_matrix.detach().cpu().numpy().
F24:1949+ astype(float).tolist(),
F24:1950             "global_orient_delta_degrees": float(global_orient_delta_degrees),
F24:1951             "global_orient_mesh_consistency_max_error_m": (
F24:1952                 global_orient_mesh_consistency_max_error_m
F24:1953             ),
F24:1954             "global_orient_stage_mean_error_px": (
F24:1955                 global_orient_stage_mean_error_px
F24:1956             ),
F24:1957             "camera_rotation_delta": camera_rotation_delta.detach().cpu().numpy().astype(float)
F24:1957+ at).tolist(),
F24:1958             "camera_refined_from_positionnet": bool(camera_fit is not None),
F24:1959             "camera_fit_mode": args.camera_fit_mode if camera_fit is not None else None,
F24:1960             "projection_consistency_max_error_px": projection_consistency_max_error_px,
F24:1961         }
F24:1962         if camera_fit is not None:
F24:1963             meta["camera_fit_joint_indices"] = camera_fit["joint_indices"].astype(int).tolist()
F24:1963+ t()
F24:1964             meta["camera_fit_initial_mean_error_px"] = camera_fit["initial_mean_error_px"]
F24:1965             meta["camera_fit_refined_mean_error_px"] = camera_fit["refined_mean_error_px"]
F24:1966             (meta_dir / f"{image_path.stem}_{bbox_id:02d}.json").write_text(json.dumps(meta, indent=2) + "\n")
F24:1966+ ent=2) + "\n")
F24:1967             persons_projected += 1
F24:1968             total_persons += 1
F24:1969             torch.cuda.empty_cache()
F24:1971             out_img = overlay_dir / f"{image_path.stem}_det_mesh_overlay.jpg"
F24:1972             cv2.imwrite(str(out_img), rgb_to_bgr_u8(vis_img))
F24:1973             saved_images.append(out_img)
F24:1974             if wireframe_img is not None:
F24:1975                 wireframe_out = wireframe_dir / f"{image_path.stem}_det_mesh_wireframe.jpg"
F24:1976                 cv2.imwrite(str(wireframe_out), rgb_to_bgr_u8(wireframe_img))
F24:1977                 saved_wireframe_images.append(wireframe_out)
F24:1978             if video_writer is not None:
F24:1979                 video_writer.write(rgb_to_bgr_u8(vis_img))
F24:1980             records.append(**det_record, "persons_projected": int(persons_projected))
F24:1982             if video_writer is not None:
F24:1983                 video_writer.release()
F24:1984             save_contact_sheet(saved_images, output_dir / "contact_sheet.jpg", max_images=args.contact_sheet
F24:1984+ _limit)
F24:1985             if saved_wireframe_images:
F24:1986                 save_contact_sheet(
F24:1987                     saved_wireframe_images,
F24:1988                     output_dir / "contact_sheet_wireframe.jpg",
F24:1989                     max_images=args.contact_sheet_limit,

```

```
F24:1990         )
F24:1991     if saved_sampling_grid_images:
F24:1992         save_contact_sheet(
F24:1993             saved_sampling_grid_images,
F24:1994             output_dir / "contact_sheet_vit_sampling_grid.jpg",
F24:1995             max_images=args.contact_sheet_limit,
F24:1996         )
F24:1997     if saved_patch_atlas_images:
F24:1998         save_contact_sheet(
F24:1999             saved_patch_atlas_images,
F24:2000             output_dir / "contact_sheet_vit_patch_atlas.jpg",
F24:2001             max_images=args.contact_sheet_limit,
F24:2002         )
F24:2003     if saved_contiguous_patch_grid_images:
F24:2004         save_contact_sheet(
F24:2005             saved_contiguous_patch_grid_images,
F24:2006             output_dir / "contact_sheet_vit_patch_atlas_grid.jpg",
F24:2007             max_images=args.contact_sheet_limit,
F24:2008         )
F24:2009     if saved_undistorted_images:
F24:2010         save_contact_sheet(
F24:2011             saved_undistorted_images,
F24:2012             output_dir / "contact_sheet_global_undistorted.jpg",
F24:2013             max_images=args.contact_sheet_limit,
F24:2014         )
F24:2015     if saved_undistorted_detection_images:
F24:2016         save_contact_sheet(
F24:2017             saved_undistorted_detection_images,
F24:2018             output_dir / "contact_sheet_global_undistorted_detection.jpg",
F24:2019             max_images=args.contact_sheet_limit,
F24:2020         )
F24:2021     if saved_positionnet_images:
F24:2022         save_contact_sheet(
F24:2023             saved_positionnet_images,
F24:2024             output_dir / "contact_sheet_positionnet.jpg",
F24:2025             max_images=args.contact_sheet_limit,
F24:2026         )
F24:2027     if saved_camera_fit_images:
F24:2028         save_contact_sheet(
F24:2029             saved_camera_fit_images,
F24:2030             output_dir / "contact_sheet_camera_fit.jpg",
F24:2031             max_images=args.contact_sheet_limit,
F24:2032         )
F24:2033     if saved_joint_compare_images:
F24:2034         save_contact_sheet(
F24:2035             saved_joint_compare_images,
F24:2036             output_dir / "contact_sheet_positionnet_vs_smplx_2d.jpg",
F24:2037             max_images=args.contact_sheet_limit,
F24:2038         )
F24:2039     if saved_mesh_joint_compare_images:
```

```

F24:2040         save_contact_sheet(
F24:2041             saved_mesh_joint_compare_images,
F24:2042             output_dir / "contact_sheet_mesh_positionnet_smplx_2d.jpg",
F24:2043             max_images=args.contact_sheet_limit,
F24:2044         )
F24:2046     summary = {
F24:2047         "image_dir": str(args.image_dir.resolve()),
F24:2048         "image_glob": args.image_glob,
F24:2049         "processed_images": int(len(image_paths)),
F24:2050         "images_without_detection": int(no_detection),
F24:2051         "projected_persons": int(total_persons),
F24:2052         "multi_person": bool(args.multi_person),
F24:2053         "detector_enabled": not args.no_detector,
F24:2054         "score_thr": float(args.score_thr),
F24:2055         "bbox_thr": float(args.bbox_thr),
F24:2056         "smplx_input_mode": args.smplx_input_mode,
F24:2057         "det_tangent_expand": float(args.det_tangent_expand),
F24:2058         "det_tangent_patch_size": float(args.det_tangent_patch_size),
F24:2059         "global_undistort_alpha": float(args.global_undistort_alpha),
F24:2060         "contiguous_roi_mode": (
F24:2061             args.contiguous_roi_mode
F24:2062             if args.smplx_input_mode == "det_undistort_contiguous"
F24:2063             else None
F24:2064         ),
F24:2065         "global_undistorted_source_k": (
F24:2066             undistort_model["source_k"].astype(float).tolist()
F24:2067             if undistort_model is not None
F24:2068             else None
F24:2069         ),
F24:2070         "global_undistorted_new_k": (
F24:2071             undistort_model["new_k"].astype(float).tolist()
F24:2072             if undistort_model is not None
F24:2073             else None
F24:2074         ),
F24:2075         "global_undistorted_distortion": (
F24:2076             undistort_model["distortion"].astype(float).tolist()
F24:2077             if undistort_model is not None
F24:2078             else None
F24:2079         ),
F24:2080         "global_undistorted_valid_roi": (
F24:2081             list(undistort_model["valid_roi"])
F24:2082             if undistort_model is not None
F24:2083             else None
F24:2084         ),
F24:2085         "input_crop_mode": args.input_crop_mode,
F24:2086         "horizontal_crop_pixels": int(args.horizontal_crop_pixels),
F24:2087         "camera_refined_from_positionnet": bool(args.refine_camera_from_positionnet),
F24:2088         "camera_fit_mode": args.camera_fit_mode,
F24:2089         "camera_fit_iters": int(args.camera_fit_iters),
F24:2090         "camera_fit_lr": float(args.camera_fit_lr),

```

```

F24:2091         "save_wireframe_overlay": bool(args.save_wireframe_overlay),
F24:2092         "projection_consistency_max_error_px": (
F24:2093             float(max(projection_consistency_errors))
F24:2094             if projection_consistency_errors
F24:2095             else None
F24:2096         ),
F24:2097         "output_dir": str(output_dir.resolve()),
F24:2098         "fisheye_calibration": str(args.fisheye_calibration.resolve()),
F24:2099         "checkpoint_path": str(args.checkpoint_path.resolve()),
F24:2100         "det_config": str(args.det_config.resolve()),
F24:2101         "det_checkpoint": str(args.det_checkpoint.resolve()),
F24:2102         "elapsed_sec": float(time.time() - start_time),
F24:2103     }
F24:2104     (output_dir / "manifest.json").write_text(json.dumps(summary, indent=2) + "\n")
F24:2105     (output_dir / "records.json").write_text(json.dumps(records, indent=2) + "\n")
F24:2106     print(json.dumps(summary, indent=2))
F24:2107     sys.stdout.flush()
F24:2108     sys.stderr.flush()
F24:2109     os._exit(0)
F24:2112 if __name__ == "__main__":
F24:2113     main()
F25:0001 """Model-free regression checks for data safety and geometric conventions."""
F25:0002 import json
F25:0003 from pathlib import Path
F25:0004 import tempfile
F25:0005 import unittest
F25:0006 import numpy as np
F25:0007 from egosmplx_pipeline.io import load, rmse, write
F25:0008 from egosmplx_pipeline.matching import assign
F25:0009 from egosmplx_pipeline.topology import boundary_edges, graph_distance_from_boundary
F25:0010 from egosmplx_pipeline.lift import native_projection, lift_mano_to_fisheye
F25:0011 from egosmplx_pipeline.configuration import load_config, expand
F25:0014 class GeometryTests(unittest.TestCase):
F25:0015     def test_rmse_is_point_distance(self):
F25:0016         self.assertEqual(rmse([[3,4]], [[0,0]]), 5)
F25:0017         self.assertIsNone(rmse(np.empty((0,2)), np.empty((0,2))))
F25:0019     def test_ring_distance_and_disconnected_mesh(self):
F25:0020         faces = np.array([[0,1,4], [1,2,4], [2,3,4], [3,0,4]])
F25:0021         self.assertEqual(len(boundary_edges(faces)), 4)
F25:0022         np.testing.assert_array_equal(graph_distance_from_boundary(faces, [0,1,2,3], 5), [0,0,0,0,1])
F25:0023         with self.assertRaises(RuntimeError):
F25:0024             graph_distance_from_boundary(faces, [0,1,2,3], 6)
F25:0026     def test_no_duplicate_hand_assignment(self):
F25:0027         observation = dict(detector_confidence=np.array([.9]), is_right=np.array([0]),
F25:0028                             joints_2d=np.tile([300.,300.], (1,21,1)))
F25:0029         lookup = {s+'_wrist':dict(x=300,y=300,score=.9) for s in ['left','right']}
F25:0030         matches = assign(observation, lookup, np.zeros((25,2)))
F25:0031         self.assertEqual(len(matches), 1)
F25:0032         self.assertEqual(matches['left']['candidate'], 0)
F25:0034     def test_missing_and_low_confidence_hands(self):

```

```

F25:0035         observation = dict(detector_confidence=np.array([.1]), is_right=np.array([1]),
F25:0036                                     joints_2d=np.tile([300.,300.], (1,21,1)))
F25:0037         self.assertEqual(assign(observation, {}, np.zeros((25,2))), {})
F25:0038         observation['detector_confidence'] = np.empty(0)
F25:0039         self.assertEqual(assign(observation, {}, np.zeros((25,2))), {})
F25:0041     def test_lift_uses_radial_distance_and_preserves_shifted_projection(self):
F25:0042         class Camera:
F25:0043             def camera2world(self, uv, radial):
F25:0044                 ray = np.c_[ (uv-[640,360])/1000, np.ones(1+uv.shape[1])]
F25:0045                 return ray/np.linalg.norm(ray,axis=1)[:,None]*radial[:,None]
F25:0046         points = np.array([[.1,.2,1.],[.2,.3,1.2]])
F25:0047         w = dict(vertices_3d_local=points[None], joints_3d_local=points[None],
F25:0048                 cam_t=np.zeros((1,3)), focal_length_px=np.array(1000.), image_size_wh=np.array([128
F25:0048+ 0,720]))
F25:0049         body_wrist = np.array([.2,.3,.7])
F25:0050         result = lift_mano_to_fisheye(Camera(), w, 0, body_wrist, np.array([800,500]))
F25:0051         expected = native_projection(points, 1000, [640,360])
F25:0052         expected += [800,500]-expected[0]
F25:0053         actual = native_projection(result['vertices'], 1000, [640,360])
F25:0054         np.testing.assert_allclose(actual, expected, atol=.0001)
F25:0055         self.assertEqual(float(np.linalg.norm(result['vertices'][0])), float(np.linalg.norm(body_wrist)), places=6)
F25:0058     class InputTests(unittest.TestCase):
F25:0059         def test_json_roundtrip_and_nonfinite_rejection(self):
F25:0060             with tempfile.TemporaryDirectory() as tmp:
F25:0061                 p=Path(tmp)/'x.json';write(p,{'中文':1})
F25:0062                 self.assertEqual(json.loads(p.read_text()), {'中文':1})
F25:0063                 q=Path(tmp)/'x.npz';np.savez(q,x=np.array([np.nan]))
F25:0064                 with self.assertRaises(ValueError):load(q)
F25:0066         def test_unresolved_variable_rejected(self):
F25:0067             with self.assertRaises(ValueError):expand('${EGOSMPLX_MISSING_TEST_VARIABLE}', Path('.'))
F25:0069         def test_path_traversal_frame_id_rejected(self):
F25:0070             with tempfile.TemporaryDirectory() as tmp:
F25:0071                 p=Path(tmp)
F25:0072                 write(p/'frames.json', {'frames':[{'id':'../outside','image':'x.jpg','calibration':'c.js
F25:0072+ on'}}])
F25:0073                 write(p/'config.json', {'paths':{},'manifest':'frames.json','output':'out'})
F25:0074                 with self.assertRaises(ValueError):load_config(p/'config.json')
F25:0077     if __name__ == '__main__':
F25:0078         unittest.main()
F26:0001     """Camera convention and differentiable projection regression tests.
F26:0003     Run in body_python. The controller-only environment may omit PyTorch.
F26:0004     """
F26:0005     import json
F26:0006     from pathlib import Path
F26:0007     import tempfile
F26:0008     import unittest
F26:0009     import numpy as np
F26:0010     try:
F26:0011         import torch

```

```

F26:0012 except ImportError:
F26:0013     torch = None
F26:0016 @unittest.skipIf(torch is None, 'PyTorch is installed in the configured body worker environment')
F26:0017 class CameraTests(unittest.TestCase):
F26:0018     def make_camera(self, **changes):
F26:0019         from egosmplx_pipeline.camera import FishEyeCameraCalibrated
F26:0020         calibration = dict(size=[1280,720], polynomialC2W=[-100.], polynomialW2C=[150.,80.,2.],
F26:0021                             image_center=[630,350], affine=[1.02,.01,-.02],
F26:0022                             camera_axis_transform=[[0,-1,0],[1,0,0],[0,0,1]])
F26:0023         calibration.update(changes)
F26:0024         with tempfile.TemporaryDirectory() as tmp:
F26:0025             path=Path(tmp)/'camera.json';path.write_text(json.dumps(calibration))
F26:0026             return FishEyeCameraCalibrated(path)
F26:0028     def test_numpy_and_horner_projection_match(self):
F26:0029         from egosmplx_pipeline.projection import stable_world2camera
F26:0030         camera=self.make_camera()
F26:0031         points=np.array([[.1,.2,.8],[-.2,.1,.4],[.1,-.2,.7]],np.float64)
F26:0032         tensor=torch.tensor(points, dtype=torch.float64,requires_grad=True)
F26:0033         pixels=stable_world2camera(camera,tensor)
F26:0034         np.testing.assert_allclose(pixels.detach().numpy(),camera.world2camera(points),atol=1e-10)
F26:0035         pixels.square().sum().backward()
F26:0036         self.assertTrue(torch.isfinite(tensor.grad).all())
F26:0038     def test_inverse_depth_is_radial_with_axis_transform(self):
F26:0039         camera=self.make_camera()
F26:0040         pixels=np.array([[600.,400.],[700.,300.]])
F26:0041         radii=np.array([.4,.8])
F26:0042         xyz=camera.camera2world(pixels,radii)
F26:0043         np.testing.assert_allclose(np.linalg.norm(xyz,axis=1),radii,rtol=1e-6)
F26:0045     def test_singular_affine_rejected(self):
F26:0046         with self.assertRaises(ValueError):self.make_camera(affine=[1,1,1])
F26:0049 if __name__ == '__main__':
F26:0050     unittest.main()
F27:0001 #!/usr/bin/env python3
F27:0002 """Export editable DOCX and paginated PDF from the actual repository sources.
F27:0004 The full source is always retained. The deposit copy uses the first and last
F27:0005 30 pages when the full listing exceeds 60 pages. No unrelated filler code is
F27:0006 generated. Applicant facts are not inferred from the GitHub account.
F27:0007 """
F27:0008 import argparse
F27:0009 import hashlib
F27:0010 import json
F27:0011 from pathlib import Path
F27:0012 import re
F27:0014 ROOT = Path(__file__).resolve().parents[1]
F27:0017 def digest(path):
F27:0018     return hashlib.sha256(path.read_bytes()).hexdigest()
F27:0021 def chunks(items, count):
F27:0022     return [items[i:i+count] for i in range(0, len(items), count)]
F27:0025 def font_setup(font_path):
F27:0026     from reportlab.pdfbase import pdfmetrics

```



```

F27:0027     from reportlab.pdfbase.ttf fonts import TTFont
F27:0028     candidates = [font_path] if font_path else [
F27:0029         Path('/usr/share/fonts/truetype/arphic-gbsn001p/gbsn001p.ttf'),
F27:0030         Path('/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf'),
F27:0031     ]
F27:0032     path = next((p for p in candidates if p and p.is_file()), None)
F27:0033     if path is None:
F27:0034         raise FileNotFoundError('Provide —font-path with a Chinese-capable TTF font')
F27:0035     pdfmetrics.registerFont(TTFont('EgoChinese', str(path)))
F27:0036     return path
F27:0039 def wrap(text, size, width):
F27:0040     """Wrap only the printed representation; source files remain unchanged."""
F27:0041     from reportlab.pdfbase.pdfmetrics import stringWidth
F27:0042     lines, line = [], ''
F27:0043     for char in text.expandtabs(4):
F27:0044         if line and stringWidth(line + char, 'EgoChinese', size) > width:
F27:0045             lines.append(line)
F27:0046             line = ''
F27:0047             line += char
F27:0048     if line:
F27:0049         lines.append(line)
F27:0050     return lines
F27:0053 def source_listing():
F27:0054     preferred = ['egosmplx_pipeline/__init__.py', 'egosmplx_pipeline/__main__.py',
F27:0055                 'egosmplx_pipeline/configuration.py', 'egosmplx_pipeline/pipeline.py',
F27:0056                 'egosmplx_pipeline/frame.py', 'egosmplx_pipeline/bootstrap.py',
F27:0057                 'egosmplx_pipeline/io.py', 'egosmplx_pipeline/sampling.py',
F27:0058                 'egosmplx_pipeline/sapiens_pose.py', 'egosmplx_pipeline/sapiens_seg.py',
F27:0059                 'egosmplx_pipeline/wilor_predict.py', 'egosmplx_pipeline/body_common.py',
F27:0060                 'egosmplx_pipeline/body_rigid.py', 'egosmplx_pipeline/body_pose.py',
F27:0061                 'egosmplx_pipeline/contour.py', 'egosmplx_pipeline/projection.py',
F27:0062                 'egosmplx_pipeline/matching.py', 'egosmplx_pipeline/lift.py',
F27:0063                 'egosmplx_pipeline/topology.py', 'egosmplx_pipeline/fusion.py',
F27:0064                 'egosmplx_pipeline/render.py', 'egosmplx_pipeline/report.py']
F27:0065     discovered = sorted(str(p.relative_to(ROOT)) for folder in ['egosmplx_pipeline', 'tools', 'tests
F27:0065+ ']
F27:0066                         for p in (ROOT / folder).rglob('*.py'))
F27:0067     order = preferred + [name for name in discovered if name not in preferred] + ['run_example.sh']
F27:0068     inventory, listing = [], []
F27:0069     for number, name in enumerate(order, 1):
F27:0070         path = ROOT / name
F27:0071         lines = path.read_text(encoding='utf-8').splitlines()
F27:0072         entry = dict(file_id='F%02d' % number, path=name, sha256=digest(path),
F27:0073                     physical_lines=len(lines), nonempty_lines=sum(bool(x.strip()) for x in lines))
F27:0074         inventory.append(entry)
F27:0075         for line_number, text in enumerate(lines, 1):
F27:0076             if not text.strip():
F27:0077                 continue
F27:0078             # Prefix allows exact correspondence after a long source line wraps.
F27:0079             prefix = '%s:%04d ' % (entry['file_id'], line_number)

```

```

F27:0080         parts = wrap(text, 9, 451)
F27:0081         for continuation, part in enumerate(parts):
F27:0082             label = prefix if not continuation else prefix.rstrip() + '+'
F27:0083             listing.append((label + part, name))
F27:0084     return inventory, chunks(listing, 50)
F27:0087 def clean_markdown(text):
F27:0088     text = re.sub(r'!\[[^\]]*\]\([^\)]*\)', '', text)
F27:0089     text = re.sub(r'\[[^\]]+\]\([^\)]+\)', r'\1 (\2) ', text)
F27:0090     text = text.replace('***', '').replace('`', '')
F27:0091     return text
F27:0094 def prose_pages(path):
F27:0095     rows = []
F27:0096     for line in clean_markdown(path.read_text(encoding='utf-8')).splitlines():
F27:0097         if not line.strip():
F27:0098             continue
F27:0099         line = re.sub(r'^#\s*', '', line)
F27:0100         rows.extend((part, str(path.relative_to(ROOT))) for part in wrap(line, 11, 505))
F27:0101     return chunks(rows, 40)
F27:0104 def export_docx(path, title, subtitle, pages, source=False, original_numbers=None, figures=()):
F27:0105     from docx import Document
F27:0106     from docx.shared import Inches, Pt
F27:0107     from docx.oxml import OxmlElement
F27:0108     from docx.oxml.ns import qn
F27:0109     document = Document()
F27:0110     section = document.sections[0]
F27:0111     section.page_width, section.page_height = Inches(8.2677), Inches(11.6929)
F27:0112     section.top_margin, section.bottom_margin = Pt(50), Pt(42)
F27:0113     section.left_margin, section.right_margin = Pt(42), Pt(42)
F27:0114     section.header_distance, section.footer_distance = Pt(18), Pt(18)
F27:0115     normal = document.styles['Normal']
F27:0116     normal.font.name = 'SimSun'
F27:0117     normal._element.rPr.rFonts.set(qn('w: eastAsia'), 'SimSun')
F27:0118     normal.font.size = Pt(9 if source else 11)
F27:0119     normal.paragraph_format.space_after = Pt(0)
F27:0120     normal.paragraph_format.space_before = Pt(0)
F27:0121     normal.paragraph_format.line_spacing = Pt(14.2 if source else 17.3)
F27:0122     header = section.header.paragraphs[0]
F27:0123     header.text = title + ' | ' + subtitle
F27:0124     for run in header.runs:
F27:0125         run.font.size = Pt(8)
F27:0126     footer = section.footer.paragraphs[0]
F27:0127     footer.alignment = 2
F27:0128     footer.add_run('第 ')
F27:0129     field = OxmlElement('w:fldSimple')
F27:0130     field.set(qn('w:instr'), 'PAGE')
F27:0131     footer._p.append(field)
F27:0132     footer.add_run(' 页')
F27:0133     for page_index, page in enumerate(pages):
F27:0134         if page_index:
F27:0135             document.add_page_break()

```

```

F27:0136         for line, _ in page:
F27:0137             paragraph = document.add_paragraph()
F27:0138             paragraph.add_run(line)
F27:0139         for caption, image in figures:
F27:0140             document.add_page_break()
F27:0141             document.add_paragraph(caption)
F27:0142             document.add_picture(str(image), width=Inches(7.0))
F27:0143         document.save(path)
F27:0146 def export_pdf(path, title, subtitle, pages, source=False, original_numbers=None, figures=()):
F27:0147     from reportlab.pdfgen.canvas import Canvas
F27:0148     from reportlab.lib.pagesizes import A4
F27:0149     from reportlab.lib.utils import ImageReader
F27:0150     canvas = Canvas(str(path), pagesize=A4)
F27:0151     canvas.setTitle(title + ' ' + subtitle)
F27:0152     canvas.setAuthor('EgoSmplx project: applicant facts pending verification')
F27:0153     width, height = A4
F27:0154     total = len(pages) + len(figures)
F27:0155     for index, page in enumerate(pages):
F27:0156         canvas.setFont('EgoChinese', 8)
F27:0157         canvas.drawString(42, height-28, title)
F27:0158         canvas.drawString(42, height-41, subtitle)
F27:0159         label = '第 %d / %d 页' % (index+1, total)
F27:0160         if original_numbers is not None:
F27:0161             label += '; 完整源码第 %d 页' % original_numbers[index]
F27:0162         canvas.drawRightString(width-42, 24, label)
F27:0163         canvas.setFont('EgoChinese', 9 if source else 11)
F27:0164         spacing = 14.2 if source else 17.3
F27:0165         for line_index, (line, _) in enumerate(page):
F27:0166             canvas.drawString(42, height-64-line_index*spacing, line)
F27:0167         canvas.showPage()
F27:0168     for number, (caption, image) in enumerate(figures, len(pages)+1):
F27:0169         canvas.setFont('EgoChinese', 8)
F27:0170         canvas.drawString(42, height-28, title)
F27:0171         canvas.setFont('EgoChinese', 11)
F27:0172         canvas.drawString(42, height-62, caption)
F27:0173         reader = ImageReader(str(image))
F27:0174         iw, ih = reader.getSize()
F27:0175         scale = min((width-84)/iw, (height-130)/ih)
F27:0176         canvas.drawImage(reader, 42, height-84-ih*scale, iw*scale, ih*scale)
F27:0177         canvas.setFont('EgoChinese', 8)
F27:0178         canvas.drawRightString(width-42, 24, '第 %d / %d 页' % (number, total))
F27:0179         canvas.showPage()
F27:0180     canvas.save()
F27:0183 def main():
F27:0184     parser = argparse.ArgumentParser(description=__doc__)
F27:0185     parser.add_argument('--output', type=Path, default=ROOT/'copyright/generated')
F27:0186     parser.add_argument('--font-path', type=Path)
F27:0187     args = parser.parse_args()
F27:0188     font_setup(args.font_path)
F27:0189     args.output.mkdir(parents=True, exist_ok=True)

```

```

F27:0190     metadata = json.loads((ROOT/'software.json').read_text())
F27:0191     title = metadata['name_zh'] + ' ' + metadata['registration_version']
F27:0192     inventory, full_pages = source_listing()
F27:0193     full_numbers = list(range(1, len(full_pages)+1))
F27:0194     selected = list(range(len(full_pages))) if len(full_pages) <= 60 else list(range(30)) + list(ran
F27:0194+ ge(len(full_pages)-30, len(full_pages)))
F27:0195     deposit = [full_pages[i] for i in selected]
F27:0196     for suffix, pages, numbers in [('完整', full_pages, full_numbers), ('交存', deposit, [i+1 for i
F27:0196+ in selected])]:
F27:0197         stem = args.output / ('03_源程序文档_' + suffix)
F27:0198         export_pdf(stem.with_suffix('.pdf'), title, '源程序文档 (' + suffix + ') ', pages, True, num
F27:0198+ bers)
F27:0199         export_docx(stem.with_suffix('.docx'), title, '源程序文档 (' + suffix + ') ', pages, True, n
F27:0199+ umbers)
F27:0200     figures = [('图1 八帧三阶段输出（左：原始；中：身体校正；右：固定缝合）', ROOT/'docs/assets/cont
F27:0200+ act_sheet.jpg')]
F27:0201     documents = [('01_登记信息填表稿', ROOT/'copyright/01_登记信息填表稿.md', []),
F27:0202                     ('02_软件设计说明书', ROOT/'docs/DESIGN_zh.md', figures),
F27:0203                     ('04_权属核对与提交清单', ROOT/'copyright/04_权属核对与提交清单.md', [])]
F27:0204     for name, path, illustrations in documents:
F27:0205         pages = prose_pages(path)
F27:0206         export_pdf(args.output/(name+'.pdf'), title, name[3:], pages, figures=illustrations)
F27:0207         export_docx(args.output/(name+'.docx'), title, name[3:], pages, figures=illustrations)
F27:0208     index = dict(software=title, files=inventory,
F27:0209                 physical_source_lines=sum(x['physical_lines'] for x in inventory),
F27:0210                 nonempty_source_lines=sum(x['nonempty_lines'] for x in inventory),
F27:0211                 printed_source_lines=sum(len(p) for p in full_pages), full_source_pages=len(full_pa
F27:0211+ ges),
F27:0212                 deposited_original_pages=[i+1 for i in selected], source_lines_per_page=50,
F27:0213                 listing_rule='Nonempty actual source lines only; long lines wrap for print with fil
F27:0213+ e/line references; no filler',
F27:0214                 ownership_status='Applicant must verify rights and third-party boundaries')
F27:0215     (args.output/'source_inventory.json').write_text(json.dumps(index,indent=2,ensure_ascii=False)+'
F27:0215+ \n')
F27:0216     (ROOT/'docs/SOURCE_INDEX.md').write_text('# 源码索引\n\n' + '\n'.join(
F27:0217         '- %s: `%s` (%d 行, 非空 %d 行) ' % (x['file_id'],x['path'],x['physical_lines'],x['nonempty_
F27:0217+ lines']) for x in inventory) + '\n')
F27:0218     print(json.dumps({k:index[k] for k in ['physical_source_lines','nonempty_source_lines','printed_
F27:0218+ source_lines','full_source_pages']},ensure_ascii=False))
F27:0221 if __name__ == '__main__':
F27:0222     main()
F28:0001 #!/usr/bin/env bash
F28:0002 set -euo pipefail
F28:0003 cd -- "$dirname -- "${BASH_SOURCE[0]}"
F28:0004 exec "${EGOSMPLX_CONTROLLER_PYTHON:-python3}" -m egosmplx_pipeline "${1:-check}" --config "${EGOSMPL
F28:0004+ X_CONFIG_FILE:-configs/pipeline.example.json}" "${@:2}"

```